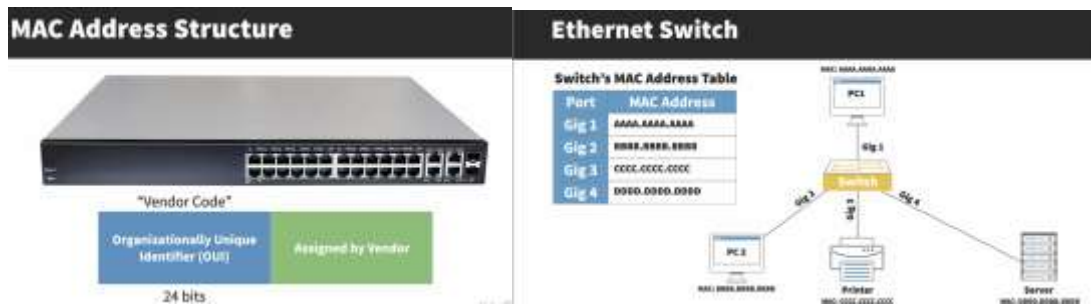


Q. What is a mac-address and why it is called burned-in-address?



Preamble: Alerts sending of an ethernet frame (7 Bytes)

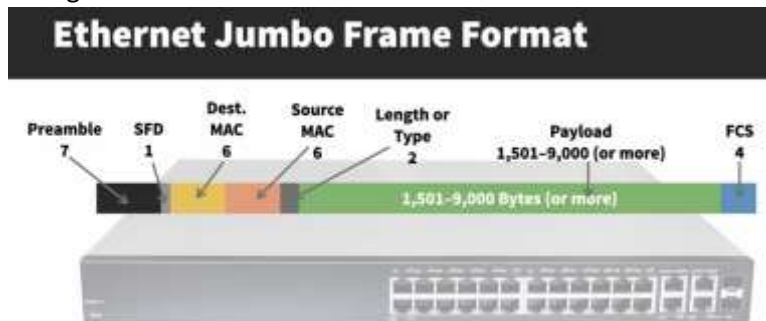
SFD (Start Frame Delimiter): Signals the end of the preamble or (1 Byte)

Dest Mac: Destination mac address (6 Bytes)

Src Mac: Source mac address (6 Bytes)

Length or Type: Specifies the size of the payload (if the value 1500 or less) or the type of network layer protocol encapsulated (if the value is 1536 Bytes or higher)

Payload: layer 3 packet containing data to be transmitted.



Q. How much overhead does an ethernet header add?

ANS: An ethernet header adds extra 18 bytes.

Collision Domain:

A collision domain is a network segment where data packets can collide if two or more devices attempt to transmit data at the same time.

1. Early Ethernet (Thinnet & Thicknet)

- Used **shared coaxial cables** with T-connectors.
- All devices on the same cable segment → **single collision domain**.
- If two devices transmit at once → **collision occurs**, data becomes garbled.

2. CSMA/CD (Carrier Sense Multiple Access with Collision Detection)

- Method to control access on shared Ethernet.
- Devices **listen** before sending.
- If channel is busy → wait until free.
- If collision happens → both stop, wait random time, then retry.
- Operates in **half-duplex** (cannot send & receive simultaneously).

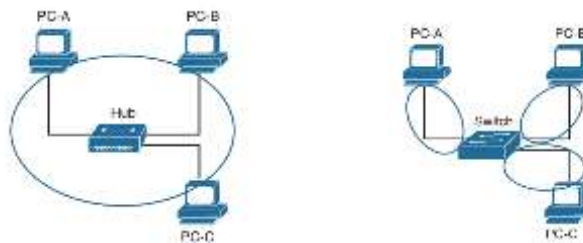
3. Problems with Shared Media

- More devices = **more collisions** = less efficiency.
- **Hubs** (multiport repeaters):
 - Increase port density, but still **one large collision domain**.
 - Simply **repeat traffic out of every port** (no intelligence).

4. Switches – The Fix

- Switches use a **MAC address table** (learns MAC → port mapping).
- Forward frames only to the correct port instead of flooding.
- Each switch port = **separate collision domain**.
- Enables **full-duplex** (simultaneous transmit & receive).
- Improves **scalability, stability, and efficiency**.

5. Example – Hub vs Switch



i. Hub:

- PC-A → PC-B traffic is also received by PC-C.
- PC-C must process & discard frames not meant for it.
- PC-C must wait for PC-A/B communication to finish before sending.
- One collision domain shared by all.

ii. Switch:

- PC-A ↔ PC-B traffic goes only between their ports.
- PC-C is not affected.
- Each port is its **own collision domain**.
- Allows multiple conversations at the same time.

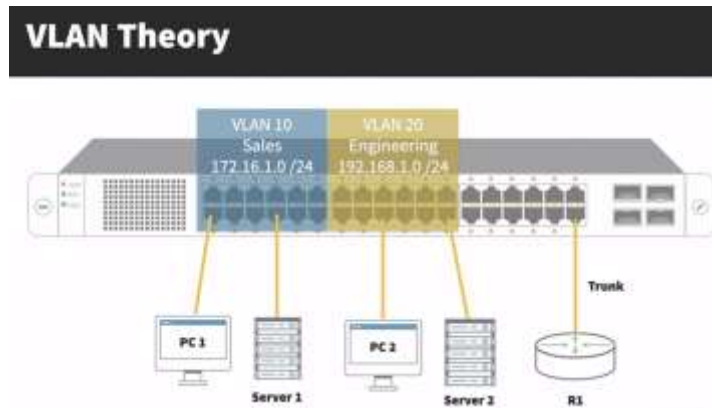
Unknown Unicast Flooding:

When a packet contains a destination MAC address that is not in the switch's MAC address table, the switch forwards the packet out of every switch port. This is known as unknown unicast flooding because the destination MAC address is not known.

It happens only if the MAC address table is cleared from the Switch and the devices that were recently pinged still have each other's MAC address in their ARP table.

VLAN (Virtual Local Area Network):

DEF: An area of network where broadcast will propagate.



Q. is VLAN layer 2 or layer3 function??

Remember, though VLANs provide separation for Layer-3 broadcast domains, they are still a Layer-2 function.

A VLAN often has a direct relationship with an IP subnet, though this is not a requirement.

vlan 0 – indicates the 802.1p header (no VLAN, but QoS priority applies)

vlan 1 – the default VLAN and cannot be modified or deleted.

VLANs 2 to 1001 – the normal VLAN range and can be added, deleted, or modified as necessary.

VLANs 1002 to 1005 – reserved and cannot be deleted (fddi, token ring).

VLANs 1006 to 4094 – the extended VLAN range and can be added, deleted, or modified as necessary

VLAN 4095 – Virtualization trunking - In **VMware ESXi**, VLAN 4095 is used for **VLAN trunking** inside a virtual switch (vSwitch/port group). This allows a VM network adapter to see all VLANs (a “trunk” port) and handle VLAN tags itself.

VLAN Creation:

vlan <vlan-id>

name <vlan-name>

show vlan Command Options:

1. show vlan brief ---- Shows only VLAN ID, VLAN name, and associated ports.

Quick overview of VLAN-to-port mappings.

```
Switch#show vlan br
VLAN Name                Status Ports
-----
1    default                active Gi0/3, Gi1/0, Gi1/1, Gi1/2
10   sales                   active Gi0/1
20   tech                    active Gi0/0
30   VLAN0030                 active
1002 fddi-default           act/unsup
1003 token-ring-default    act/unsup
```

2. show vlan summary

Displays: Total number of VLANs.

Number of VLANs in VTP domain.

VLANs in the extended range (1006–4094).

```
Switch#show vlan summary
Number of existing VLANs           : 9
Number of existing VTP VLANs       : 8
Number of existing extended VLANs  : 1
```

3. show vlan id <vlan-id>

Detailed output for a specific VLAN number.
Includes VLAN name, status, and ports assigned.

```
Switch#show vlan id 10
```

VLAN Name	Status	Ports
10 sales	active	Gi0/1, Gi0/2

VLAN Type	SAID	MTU	Parent RingNo	BridgeNo	Stp	BrdgMode	Trans1	Trans2
10	enet	100010	1500	-	-	-	0	0


```
Remote SPAN VLAN
-----
Disabled
```


Primary	Secondary	Type	Ports

4. show vlan name <vlan-name>

Same as above, but filtered by VLAN name instead of ID.

```
Switch#show vlan name sales
```

VLAN Name	Status	Ports
10 sales	active	Gi0/1, Gi0/2

VLAN Type	SAID	MTU	Parent RingNo	BridgeNo	Stp	BrdgMode	Trans1	Trans2
10	enet	100010	1500	-	-	-	0	0


```
Remote SPAN VLAN
-----
Disabled
```


Primary	Secondary	Type	Ports

VLANs provide the several benefits:

- **Broadcast Control** – eliminates unnecessary broadcast traffic, improving network performance and scalability.
- **Security** – logically separates users and departments, allowing administrators to implement access-lists to control traffic between VLANs.
- **Flexibility** – removes the physical boundaries of a network, allowing a user or device to exist anywhere.

VLAN Membership can be configured one of two ways:

- Statically (manually assign the ports to a particular VLAN)
- Dynamically (bind the mac-address to a particular VLAN)

VLAN Port Types A VLAN-enabled switch supports two types of ports:

- Access ports (shares a single vlan information)
- Trunk ports (shares multiple vlan information)

Static vlan & Access port configuration:

```
interface g0/1
name technical team m-shan
switchport mode access
switchport access vlan 10          (or switchport access vlan name sales)
```

Trunk Port configuration:

```
int g0/2
switchport trunk encapsulation dot1q
switchport mode trunk
```

```
Switch#show interface trunk
```

Port	Mode	Encapsulation	Status	Native vlan
Gi0/2	on	802.1q	trunking	1


```
Port      Vlans allowed on trunk
Gi0/2     1-4094
```



```
Port      Vlans allowed and active in management domain
Gi0/2     1,10,20,30,3007
```



```
Port      Vlans in spanning tree forwarding state and not pruned
Gi0/2     1,10,20,30,3007
```

(show interface trunk -- it shows native vlan, displays all of the VLANs that are allowed to be transmitted)

across ! the trunk ports, and displays all of the VLANs that are allowed across the trunk and are ! in a spanning tree forwarding state)

VLAN Frame-Tagging

When VLANs span multiple switches, a mechanism is required to identify which VLAN a frame belongs to. This is accomplished through frame tagging, which places a VLAN ID in each frame. Tagging only occurs when a frame is sent out a trunk port. Traffic sent out access ports is never tagged. Consider the following example:

Frame Tagging Protocols Cisco switches support two frame tagging protocols:

- Inter-Switch Link (ISL)
- IEEE 802.1Q The tagging protocol can be manually specified on a trunk port, or dynamically negotiated using Cisco's proprietary Dynamic Trunking Protocol (DTP). Inter-Switch Link (ISL) Inter-Switch Link (ISL) is Cisco's proprietary frame tagging protocol.

ISL supports several technologies:

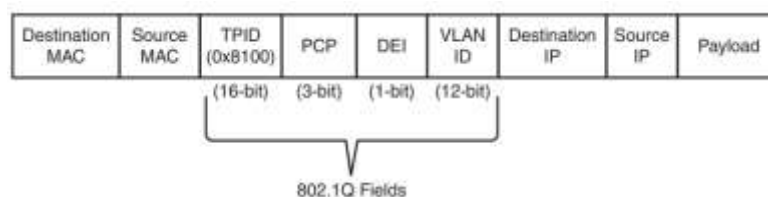
- Ethernet
- Token Ring
- FDDI
- ATM ISL encapsulates a frame with an additional header (26 bytes)

Normally, the maximum possible size of an Ethernet frame is 1518 bytes. This is known as the Maximum Transmission Unit (MTU). Most Ethernet devices use a default MTU of 1514 bytes. ISL increases the frame size by another 30 bytes. Thus, most switches will disregard ISL-tagged frames as being oversized, and drop the frame. An oversized frame is usually referred to as a giant. Somewhat endearingly, a slightly oversized frame is known as a baby giant.

IEEE 802.1Q

IEEE 802.1Q, otherwise referred to as dot1Q, is an industry-standard frame-tagging protocol. 802.1Q is supported by nearly all switch manufacturers, including Cisco. Because 802.1Q is an open standard, switches from different vendors can be trunked together. ISL encapsulates a frame with an additional header and trailer. In contrast, 802.1Q embeds a 4-byte VLAN tag directly into the Layer-2 frame header. Because the Layer-2 header is modified, 802.1Q must recalculate the frame's CRC value. The VLAN tag includes a 12-bit VLAN ID. This tag increases the size of an Ethernet frame, from its default of 1514 bytes to 1518 bytes. Nearly all modern switches support the 802.1Q tag and the slight increase in frame size. 802.1Q supports a maximum of 4096 VLANs on a trunk port.

When a frame is tagged for VLANs, **32 bits** are inserted into the Ethernet header after the Source MAC address field and before the EtherType field.



Fields inside the 802.1Q tag:

1. TPID (Tag Protocol Identifier) – 16 bits
Value: 0x8100
Identifies the frame as carrying 802.1Q VLAN information.
2. PCP (Priority Code Point) – 3 bits
Defines Class of Service (CoS) for Layer 2 QoS.
Values 0–7 (higher = higher priority).
3. DEI (Drop Eligible Indicator) – 1 bit
Indicates whether a frame can be dropped in case of congestion.
0 = Not drop eligible, 1 = Drop eligible.

4. VLAN ID (VID) – 12 bits

Identifies the VLAN the frame belongs to.

Range: 0–4095 (0 and 4095 are reserved → usable VLANs: 1–4094).

Configuring Trunk Links To manually configure an interface as a trunk port:

```
interface gi2/24
```

```
switchport trunk encapsulation isl
```

```
switchport mode trunk
```

```
interface gi2/24
```

```
switchport trunk encapsulation dot1q
```

```
switchport mode trunk
```

Native VLAN's:

In the 802.1Q standard, any traffic that is advertised or received on a trunk port without the 802.1Q VLAN tag is associated to the native VLAN.

Command to change native VLAN: `switchport trunk native vlan <vlan-id>`

Trunk port traffic engineering:

With the help of **Allowed vlan** command, we can restrict the number of vlan's allowed on trunk port.

allowed traverse on trunk: `switchport trunk allowed vlan 1,10,20 ...` (list being overwritten)

The full command syntax

switchport trunk allowed vlan { ids | all | none | add vlan-ids | remove vlan-ids | except vlan-ids }

MAC Address Table:

The MAC address table is responsible for identifying the switch ports and VLANs with which a device is associated. A switch builds the MAC address table by examining the source MAC address for traffic that it receives. This information is then maintained to shrink the collision domain (point-to-point communication between devices and switches) by reducing the amount of unknown unicast flooding.

`show mac address-table [address mac-address | dynamic | vlan vlan-id]`

3. Why static MAC helps

By configuring a **static MAC address entry**, you:

- Pin that virtual MAC to a **specific port** (the port toward the load balancer or server cluster).

- Prevent the switch from constantly relearning and flooding.

- Ensure traffic destined for the virtual MAC always goes to the correct place.

Command: `mac address-table static <mac-address> vlan <vlan-id> {drop | interface <interface-id>}`

`clear mac address-table dynamic [{address mac-address | interface interface-id | vlan vlan-id}]`

adds a manual entry with the ability to associate it to a specific switch port or to drop traffic upon receipt.

Interface Status:

```
Switch#show interfaces status
```

Port	Name	Status	Vlan	Duplex	Speed	Type
Gi0/0	"v20 - tech team"	connected	20	a-full	auto	RJ45
Gi0/1		connected	10	a-full	auto	RJ45
Gi0/2		connected	trunk	a-full	auto	RJ45
Gi0/3		connected	1	a-full	auto	RJ45
Gi1/0		connected	1	a-full	auto	RJ45
Gi1/1		connected	1	a-full	auto	RJ45
Gi1/2		connected	1	a-full	auto	RJ45
Gi1/3		connected	1	a-full	auto	RJ45

Switch Port Status:

`show interfaces interface-id switchport`

`show interfaces switchport`

(the above command all information regarding switch

for all port)

```

Switch#show interface g0/0 switchport
Name: Gi0/0
Switchport: Enabled
Administrative Mode: static access
Operational Mode: static access
Administrative Trunking Encapsulation: negotiate
Operational Trunking Encapsulation: native
Negotiation of Trunking: Off
Access Mode VLAN: 20 (tech)
Trunking Native Mode VLAN: 1 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk associations: none
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL

Protected: false
Appliance trust: none

```

INTERVLAN ROUTING:

Routed Subinterfaces (Router on a stick):

In the past, there might have been times when a switch has many VLANs but the router doesn't have enough physical ports, you can solve this by configuring the switch port as a trunk and creating sub-interfaces on the router. Each sub-interface is a virtual port and is assigned to a specific VLAN. This lets a single router interface handle routing for multiple VLANs.

command:

```

int g0/0
no sh
int g0/0.<vlan-id>      (better to use vlan-id for understanding)
encapsulation dot1q <vlan-id>
ip add <ip> <subnet-mask>

```

Switched Virtual Interfaces:

With Catalyst switches it is possible to assign an IP address to a switched virtual interface (SVI), also known as a VLAN interface. An SVI is configured by defining the VLAN on the switch and then defining the VLAN interface with the command **interface vlan <vlan-id>**. The switch must have an interface associated to that VLAN in an up state for the SVI to be in an up state. If the switch is a multilayer switch, the SVIs can be used for routing packets between VLANs without the need of an external router.

```

SW1(config)# interface vlan 10
SW1(config-if)# ip address 10.10.10.1 255.255.255.0
SW1(config-if)# ipv6 address 2001:db8:10::1/64
SW1(config-if)# no shutdown
SW1(config-if)# interface vlan 99
SW1(config-if)# ip address 10.99.99.1 255.255.255.0
SW1(config-if)# ipv6 address 2001:db8:99::1/64
SW1(config-if)# no shutdown

```


Routed Port:

Sometimes, networks use a point-to-point link between a switch and a router. In the past, engineers would create a special VLAN (like VLAN 2001) for this link, assign the switch port to that VLAN, and make a VLAN interface (SVI) for it. But this could cause problems if the same VLAN exists elsewhere in the network or if spanning tree affects the connection. A simpler way is to change the switch port from a normal Layer 2 port to a **routed port** using the command **no switchport**, so it acts like a direct router connection without needing a VLAN

```
SW1(config)# int gi 0/0
SW1(config-if)# no switchport
SW1(config-if)# ip address 10.20.20.1 255.255.255.0
SW1(config-if)# ipv6 address 2001:db8:20::1/64
SW1(config-if)# no shutdown
```

Verification of switch port:

It shows both ip assigned and unassigned ports.

```
Switch#show ip int br
Interface      IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0    unassigned      YES unset    up          up
GigabitEthernet0/1    unassigned      YES unset    up          up
GigabitEthernet0/2    unassigned      YES unset    up          up
GigabitEthernet0/3    10.1.2.1        YES manual   up          up
GigabitEthernet1/0    unassigned      YES unset    up          up
GigabitEthernet1/1    unassigned      YES unset    up          up
GigabitEthernet1/2    unassigned      YES unset    up          up
GigabitEthernet1/3    unassigned      YES unset    up          up
Vlan1           10.1.1.2        YES manual   up          up
```

It shows only ip assigned ports.

```
Switch#show ip interface brief | exclude unassigned
Interface      IP-Address      OK? Method Status      Protocol
GigabitEthernet0/3    10.1.2.1        YES manual   up          up
Vlan1           10.1.1.2        YES manual   up          up
```

For IPV6:

```
Switch#show ipv6 interface brief | exclude unassigned
GigabitEthernet0/0    [up/up]
GigabitEthernet0/1    [up/up]
GigabitEthernet0/2    [up/up]
GigabitEthernet0/3    [up/up]
FE80::5200:FF:FE02:3
2001:10::1
GigabitEthernet1/0    [up/up]
GigabitEthernet1/1    [up/up]
GigabitEthernet1/2    [up/up]
GigabitEthernet1/3    [up/up]
Vlan1                 [up/up]

Switch#show ipv6 interface brief | exclude unassigned|GigabitEthernet
FE80::5200:FF:FE02:3
2001:10::1
Vlan1                 [up/up]
```


Forwarding Architectures:

Packet Forwarding in Cisco Routers

1. Early Routers (Old Method)

Router removes the Layer 2 header (like MAC addresses).

Checks if a route exists for the destination IP.

- If no route → drop the packet.
- If route found → add a new Layer 2 header and send it out.

2. Improved Method (Modern Routers)

Instead of fully removing & adding Layer 2 info, the router just rewrites the necessary fields (faster).

This process is called **IP packet forwarding**.

Helps routers handle large volumes of traffic efficiently.

3. Packet Switching Methods in Cisco Routers

Process Switching (oldest, slowest)

- CPU handles every packet individually.

Fast Switching (improvement)

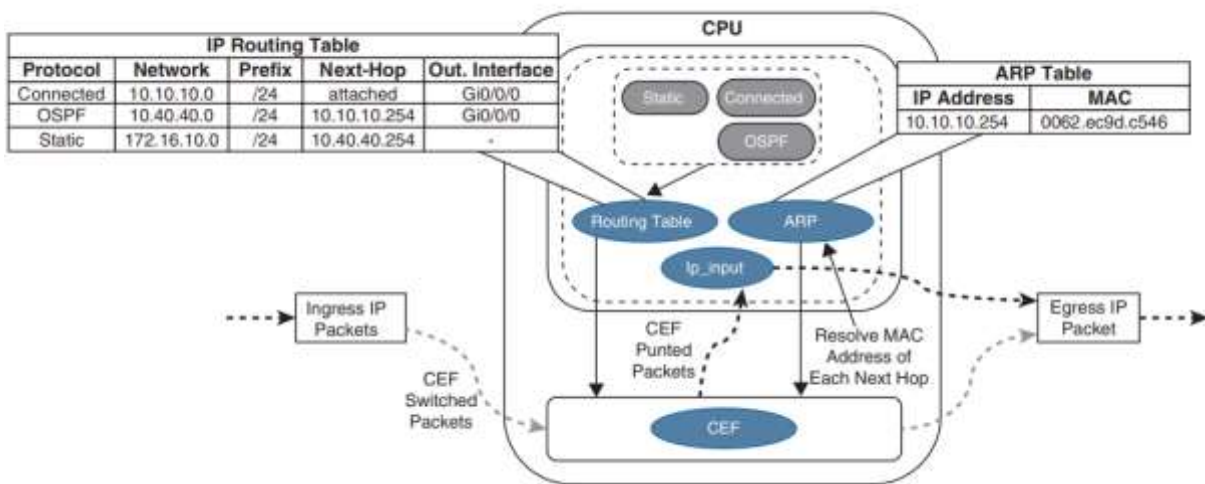
- First packet checked by CPU, result stored in cache.
- Later packets use the cache → faster than process switching.

Cisco Express Forwarding (CEF – modern, fastest)

- Pre-computed tables (**FIB + adjacency table**) (**Enterprise dump**).
- No cache lookups needed, all packets switched at hardware speed.

Final Understand:

- Old routers = remove & rebuild Layer 2 headers → slow.
- Modern routers = just rewrite Layer 2 headers → fast.
- Evolution: Process switching → Fast switching → CEF (default today)



What is CEF?

- Cisco Express Forwarding (CEF) is a fast and scalable packet-switching mechanism.
- It was created because earlier switching methods (like process switching and fast switching) couldn't keep up with modern traffic demands.
- Today, CEF is the default switching mechanism on almost all Cisco platforms.

How CEF Works

CEF separates routing (done by the control plane) from forwarding (done by the data plane):

- Control Plane (CPU / Routing Engine)
 - Builds the Routing Information Base (RIB) and Adjacency Table.
- Data Plane (Forwarding Engine: ASICs/NPUs/TCAM or CPU in software routers)
 - Uses a precomputed Forwarding Information Base (FIB) and Adjacency Table to switch packets at high speed.
 - So instead of looking up the routing table for every packet, the router precomputes the best paths and stores them in efficient hardware/software tables.

Software CEF vs Hardware CEF

- Software CEF (on software-based routers)
Everything (control plane + data plane) runs on the general-purpose CPU.
Example: Cisco ISR series.
- Hardware CEF (on hardware-based routers)
Control plane runs on the CPU, but forwarding is offloaded to:
ASICs (Application-Specific Integrated Circuits) → fixed-function, very fast.
NPUs (Network Processing Units) → programmable packet processors.
TCAM (Ternary Content-Addressable Memory) → ultra-fast lookups for ACLs, QoS, and prefix matching.
Example: Cisco Catalyst, Nexus, and ASR series.

Why CEF is Important

1. High performance → handles millions of packets per second.
2. Scalability → works in large core networks.
3. Deterministic switching → avoids route-cache lookups like in fast switching.
4. Supports advanced features → QoS, MPLS, load balancing, ACLs.

CAM vs TCAM (Ternary Content Addressable Memory):

- CAM (Content Addressable Memory)
Binary lookup only (0 or 1).
Used for exact match lookups.
Common use: MAC address table in Layer 2 switches (find output port for a destination MAC).
- TCAM (Ternary Content Addressable Memory)
Can return 0 (true), 1 (false), or **X (don't care)** pattern-based searches using a third.
Supports longest prefix match and mask-based lookups.
Used for ACLs, QoS policies, and L3 forwarding decisions.
Provides parallel lookups across multiple fields, meaning a switch can check source/destination IP, VLAN, QoS marking, protocol, etc. all at once.

TCAM Entry Format: VMR

Each TCAM entry has three parts (VMR):

1. Value → What to match (e.g., IP address, protocol, DSCP)
2. Mask → Determines which bits in the value are relevant for the lookup
3. Result → What action to take (forward, drop, redirect, police, etc.)

Example:

Value	Mask	Result
192.168.1.0	255.255.255.0	Forward out Fa0/1
Any TCP port 80	Mask "X" fields	Send to QoS policer
Deny 10.0.0.0/8	255.0.0.0	Drop packet

Why TCAM is Important

1. Speed
Hardware-based lookups → deterministic, line-rate forwarding.
ACL processing is O(1) (constant time), regardless of the number of entries (10 or 500+).
2. Multiple Simultaneous Checks
Layer 2 forwarding (MAC lookup).
Layer 3 forwarding (IP lookup).
Security (ACLs).

QoS classification/marketing.

All happen in parallel, not sequentially.

3. Scalability

Large ACLs or QoS policies don't slow down performance.

Switches can handle complex enterprise policies at line speed.

Centralized Forwarding:

- **Definition:** All packet forwarding decisions are made by the **Route Processor (RP)** or central forwarding engine.
- **Process:**
 1. Ingress line card receives packet.
 2. Forwards it to the **RP forwarding engine**.
 3. RP examines packet headers → performs lookup.
 4. Sends packet to the appropriate egress line card.
- **Performance:**

Simpler, cheaper (relies mostly on general-purpose CPU).
Limited **throughput** because all packets must pass through RP.
- **Use cases:**

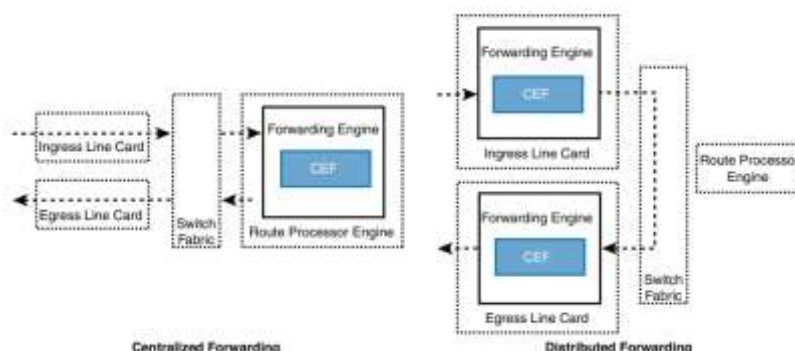
Lower-end routers (e.g., Cisco ISR series).
Networks with lighter traffic demands.

Distributed Forwarding:

- **Definition:** Each line card has its own forwarding engine (**ASIC/NPUs**) to perform lookups locally.
- **Process:**
 1. Ingress line card receives packet.
 2. Local forwarding engine performs lookup (using FIB/adjacency table copied from RP).
 3. If destination interface is local → forward directly.
 4. If destination interface is on another line card → send across **switch fabric/backplane** directly to egress line card (no RP involvement).
- **Performance:**

Highly scalable and supports high packet throughput.
Forwarding distributed across line cards, eliminating RP bottleneck.
- **Use cases:**

High-end enterprise/core routers and modular switches (e.g., Cisco ASR, CRS, Catalyst 9600, Nexus).
Large-scale, high-performance networks.



Route Processor Redundancy Modes

1. RPR (Route Processor Redundancy)

- Standby RP only has **basic configuration**.
- On switchover → Standby **reloads**, reboots routing protocols, and rebuilds adjacencies.
- Long downtime (not used much anymore).

2. SSO (Stateful Switchover)

- **Configuration + Layer 2 state** is checkpointed from Active → Standby RP.
- Line cards & interfaces remain up during switchover (no link flaps).
- Faster recovery than RPR.
- But **Layer 3 state (routing protocols)** is lost → adjacency flaps → routing table cleared → CEF purged.
- Result: **forwarding disruption** until routes are relearned.

3. NSF (Nonstop Forwarding)

- Built **on top of SSO**.
- Idea: Keep **CEF/FIB entries** active in line cards during RP failover.
- Line cards continue forwarding packets while routing protocols restart.
- Routing peers must support **GR (Graceful Restart)** to temporarily keep adjacencies.
- Data plane keeps forwarding → less disruption.
- Still some control-plane reconvergence required.

4. NSR (Nonstop Routing)

- Strongest form of redundancy.
- Unlike NSF (which needs GR-capable neighbors), NSR replicates the **entire routing protocol process state** from Active RP → Standby RP in real time.
- During failover → Standby RP already has the **full routing protocol adjacency + RIB + FIB**, so no reset occurs.
- Truly seamless switchover.
- More CPU/memory overhead.

SDM Templates:

- SDM stands for Switch Database Management or Switch Database Manager template.
- It's a crucial feature on Cisco switches (and similar concepts exist on other vendors' hardware) that allows you to optimize how the switch's limited hardware resources are allocated to different networking functions.

How does an SDM template work?

You select a pre-defined template that tells the switch how to divide up its TCAM memory between various features:

- Layer 2 MAC addresses: For switching within a VLAN.
- Layer 3 IPv4 and IPv6 routes: For routing between VLANs (Inter-VLAN routing)
- IPv4 and IPv6 ACLs (Access Control Lists): For security and traffic filtering.

- QoS (Quality of Service) entries: For classifying and prioritizing traffic.
- Multicast routes: For efficient multicast traffic handling.
- Other advanced features like DHCP snooping, IGMP groups, etc.

```
S1#show sdm prefer ?
```

```
access  Show access template configuration
default Show default template configuration
routing Show routing template configuration
vlan    Show VLAN template configuration
|       Output modifiers
<cr>
```

```
S1#show sdm prefer routing
routing template:
```

The selected template optimizes the resources in the switch to support this level of features for 8 routed interfaces and 1K VLANs.

```
number of unicast mac addresses: 5K
number of igmp groups:          1K
number of qos aces:             512
number of security aces:        512
number of unicast routes:       16K
number of multicast routes:     1K
```

command used to go back default sdm template:
(configuration)# no sdm template

STP (Spanning Tree Protocol):

It is used when redundant paths exist in a Layer 2 topology. Because of these redundant paths, loops can occur in the topology. In a Layer 3 topology, loops are discarded with the help of the TTL value. However, in Layer 2, no such mechanism exists to prevent loops. STP temporarily resolves this issue by blocking the Layer 2 loop, choosing specific ports to act as forwarding, blocking, or designated ports.

STP has divided into 5 categories. They are:

- 802.1D, which is the original specification, also known as common spanning tree
- Per-VLAN Spanning Tree (PVST)
- Per-VLAN Spanning Tree Plus (PVST+)
- 802.1W Rapid Spanning Tree Protocol (RSTP)
- 802.1S Multiple Spanning Tree Protocol (MST)

Spanning-Tree Communication: Bridge Protocol Data Units:

STP operates as switches communicate with one another. Data messages are exchanged in the form of bridge protocol data units (BPDUs). A switch sends a BPDU frame out a port, using the unique MAC address of the port itself as a source address. The switch is unaware of the other switches around it, so BPDU frames are sent with a destination address of the well-known STP multicast address **01-80-c2-00-00-00**.

Two types of BPDU exist:

- Configuration BPDU, used for spanning-tree computation
- Topology Change Notification (TCN) BPDU, used to announce changes in the network topology

Configuration BPDU contains:

Protocol ID (always 0), Version, Message Type (Configuration or TCN BPDU), Flags, Root Bridge ID, Root Path Cost, Sender Bridge ID, Port ID, Maximum age, Hello timer, Forward delay timer

Electing a Root Bridge:

The exchange of BPDU messages works toward the goal of electing reference points or root bridge as a foundation for a stable spanning-tree topology.

By default, BPDUs are sent out all switch ports every 2 seconds so that current topology information is exchanged and loops are identified quickly.

Each switch has a unique **bridge ID** that identifies it to other switches. The bridge ID is an 8-byte value consisting of the following fields:

- i. **Bridge priority** (2 bytes): The Priority field can have a value of 0 to 65,535 and defaults to 32,768 (or 0x8000) on every Catalyst switch.
- ii. **Mac address** (6 bytes):

Root bridge election criteria:

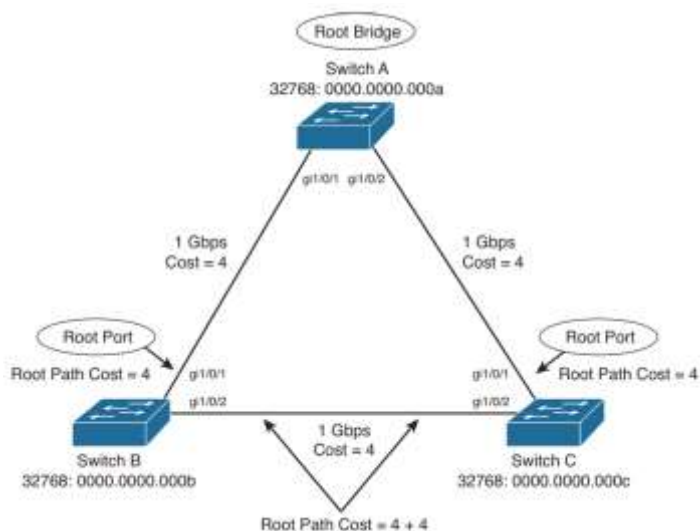
1. First, it will select the root bridge based on the priority of the switch. Lower priority switch will be elected as root bridge.
2. If Priority ties then mac address acts as tie-breaker. Lower mac-address switch will be elected as a root bridge.

Electing Root ports:

Selecting a root port involves evaluating the **root path cost**. This value is the cumulative cost of all the links leading to the root bridge. A particular switch link also has a cost associated with it, called the **path cost**. All the non-root switches must elect a Root port that is the lowest path cost to reach the root bridge.

Table 2-2 Default Interface STP Port Costs

Link Speed	Short-Mode STP Cost	Long-Mode STP Cost
10 Mbps	100	2,000,000
100 Mbps	19	200,000
1 Gbps	4	20,000
10 Gbps	2	2,000
20 Gbps	1	1,000
100 Gbps	1	200
1 Tbps	1	20
10 Tbps	1	2



PORT TYPES AND THEIR ELECTION CRITERIA:

Root port:

After the root bridge is elected, every non-root switch must choose one port as its Root Port (the best path toward the root bridge). The decision is made step by step like this:

1. The interface associated to lowest path cost is more preferred.
2. The interface associated to the lowest system priority of the advertising switch is preferred next.
3. The interface associated to the lowest system MAC address of the advertising switch is preferred next.
4. When multiple links are associated to the same switch, the lowest port priority from the advertising switch is preferred.
5. When multiple links are associated to the same switch, the lower port number from the advertising switch is preferred

Designated & Blocked Ports:

One Designated Port per segment → The port that forwards traffic *onto* the segment.

Election is based on Bridge Protocol Data Units (BPDUs) exchanged by switches.

Decision Process

1. Rule out RPs

Only consider ports that are designated. Root ports are never blocked in this process.

2. Compare path cost to root bridge

- The switch with the lower path cost keeps forwarding.
- The switch with the higher path cost blocks.

3. Compare bridge (system) priority (if costs are equal)

- The switch with the lower bridge priority (numerically smaller) keeps forwarding.
- The switch with the higher bridge priority blocks.

4. Compare bridge MAC address (if priorities are equal)

- The switch with the lower MAC address keeps forwarding.
- The switch with the higher MAC address blocks.

5. Tie within the same switch (two ports of the same device)

- If both links are on the same switch, then the lowest port ID (priority + port number) determines which port forwards, and the other blocks.

802.1D PORT STATES:

i. Disable: Ports that are turned off manually by an administrator, or automatically by the system when there is a problem, go into the Disabled state. This state is different because it is not part of the normal STP port states that a port usually goes through.

ii. Block: When a port first starts, it goes into the Blocking state to stop any network loops. In this state, the port cannot send or receive normal data, and it does not learn MAC addresses. The only thing it can do is listen to BPDUs from other switches.

iii. Listening: A port moves from Blocking to Listening if the switch thinks it might become a **Root Port** or a Designated Port. This means the port is getting ready to start forwarding traffic. In the Listening state, the port still cannot send or receive normal data, but it can send and receive BPDUs. This lets the port take part in the Spanning Tree process. If the port is chosen as a Root Port or Designated Port, it will move forward. If not, it goes back to the Blocking state.

iv. Learning: After spending some time in the Listening state (forward delay time), the port moves to the Learning state. In this state, the port still sends and receives BPDUs. It also starts to learn MAC addresses and store them in its table. This helps the switch prepare with address information. However, the port still cannot send normal data frames yet.

v. Forwarding: After spending some more time in the Learning state (another forward delay), the port moves to the Forwarding state. Now the port can send and receive normal data, continue learning MAC addresses, and also send and receive BPDUs. The port is now a fully functioning switch port within the spanning-tree topology.

command to check the state of a port: show spanning interface gigabitethernet 0/1

802.1D STP TIMERS:

STP works by having switches send BPDUs to each other so they can build a loop-free network. But it takes time for BPDUs to travel between switches, and if the network changes (like a link or root bridge fails), it also takes time for that news to spread across the network. To prevent loops while waiting for all switches to update, STP uses timers.

The three main STP timers are:

• Hello Timer:

How often the Root Bridge sends out Configuration BPDUs.

All other switches just forward these messages as they get them.

Default = **2 seconds**.

• Forward Delay Timer:

How long a port stays in the Listening and Learning states before moving to Forwarding.

Default = **15 seconds** (per state). It can be changed, it depends upon the diameter of the STP.

• Max Age Timer

It is the amount of time a switch keeps a BPDU before discarding it. Each port remembers the

best BPDU it has received. If the port stops getting BPDUs from that source, it waits for the Max Age time.

After that time passes (default **20 seconds**) with no new BPDUs, the switch assumes the network has changed and removes the old BPDU.

31	39.882638	50:00:00:04:00:00	Spanning-tree-(for- STP	60 Topology Change Notification
32	39.890255	50:00:00:01:00:01	Spanning-tree-(for- STP	60 Conf. TC + Root = 32768/1/50:00
33	40.936478	50:00:00:01:00:01	Spanning-tree-(for- STP	60 Conf. TC + Root = 32768/1/50:00
34	42.048045	50:00:00:01:00:01	Spanning-tree-(for- STP	60 Conf. TC + Root = 32768/1/50:00

```

> Frame 31: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface -, id 0
> IEEE 802.3 Ethernet
> Logical-Link Control
  > Spanning Tree Protocol
    Protocol Identifier: Spanning Tree Protocol (0x0000)
    Protocol Version Identifier: Spanning Tree (0)
    BPDU Type: Topology Change Notification (0x80)

```

TOPOLOGY CHANGES:

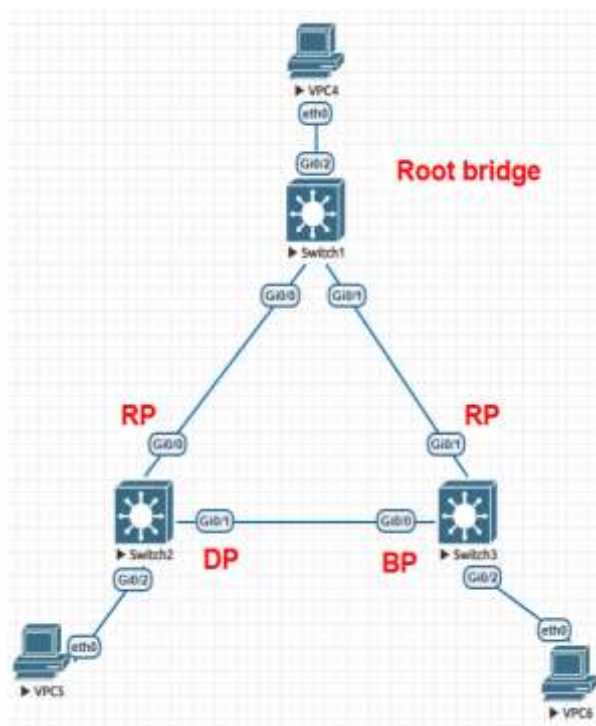
A topology change happens in STP when:

- A port moves into the Forwarding state, or
- A port moves from Forwarding/Learning into Blocking.

This simply means a port on a switch either comes up or goes down.

When this happens:

1. The switch sends a TCN BPDU (Topology Change Notification) out its Root Port.
2. The TCN does not say *what* changed—it only says *something* changed.
3. The switch keeps sending the TCN every Hello Time (2s by default) until it gets an acknowledgment from its upstream switch.
4. The TCN is passed hop by hop until the Root Bridge gets it.
5. The Root Bridge then sets the Topology Change flag in its normal Configuration BPDUs.
6. All switches receive this flag and reduce their MAC address aging time from 300s (default) to 15s (Forward Delay).
 - This makes switches quickly clear out old MAC addresses, so incorrect entries don't cause traffic issues.
 - However, active devices that are sending traffic stay in the table.
7. This topology change condition lasts for Max Age + Forward Delay (20 + 15 = 35 seconds by default).



TYPES OF TOPOLOGY CHANGES:

There are 3 types of topology changes. They are:-

1. Direct Topology Changes:

A direct topology change is one that can be detected on a switch interface. For example, if a trunk link suddenly goes down, the switch on each end of the link can immediately detect a link failure. The absence of that link changes the bridging topology, so other switches should be notified.

(topology is stable, mac-address table of sw3 shows below info)

```
Switch#show mac address-table
Mac Address Table
-----
Vlan    Mac Address      Type      Ports
----
1       5000.0001.0001   DYNAMIC   Gi0/1
1       5000.0002.0000   DYNAMIC   Gi0/1
1       5000.0003.0000   DYNAMIC   Gi0/1
Total Mac Addresses for this criterion: 3
```

```
Switch#show spanning-tree
VLAN0001
Spanning tree enabled protocol ieee
Root ID    Priority    32769
Address    5000.0001.0000
Cost       4
Port       2 (GigabitEthernet0/1)
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

Bridge ID  Priority    32769 (priority 32768 sys-id-ext 1)
Address    5000.0003.0000
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
Aging Time 300 sec

Interface Role Sts Cost Prio.Nbr Type
-----
Gi0/0    Altn BLK 4    128.1 P2p
Gi0/1    Root FWD 4    128.2 P2p
Gi0/2    Desg FWD 4    128.3 P2p
Gi0/3    Desg FWD 4    128.4 P2p
Gi1/0    Desg FWD 4    128.5 P2p
```

(Topology for example)

I applied the shutdown command on G0/1. SW3 removed all the MAC addresses from its table, and other devices could no longer ping the devices connected to SW3. Here, SW1 knows that a topology change has occurred since it is directly connected to SW3. It doesn't wait for any TCN BPDU and immediately sends a TCN BPDU using port G0/0.

```
Switch#show mac address-table
Mac Address Table
-----
Vlan    Mac Address      Type      Ports
-----
-----
-----
-----
```

The root bridge, **Switch 1**, sends a Configuration BPDU with the TCN bit set out its port g0/0. This is received and relayed by each switch along the way, informing each one of the topology.

Switches 2 and 3 receive the TCN message. The only reaction these switches take is to shorten their bridging table aging timers to the forward delay time. At this point, they do not know how the topology has changed; they only know to force fairly recent bridging table entries to age out.

Switch 3 basically just sits and waits to hear from the root bridge again. The Configuration BPDU TCN message is received on port G0/2, which was previously in the Blocking state. This BPDU becomes the best one received from the root, so port G0/2 becomes the new root port.

Q. How much time does SW3 take to move from the Blocking state to the Forwarding state?

After 30 seconds ($2 \times \text{Forward Delay}$), SW3 has two MAC addresses in its table because the blocked port G0/0 transitioned from the Blocking state to the Forwarding state. To reach Forwarding, the port first had to pass through the **Listening and Learning** states.

```
Switch#show mac address-table
Mac Address Table
-----
Vlan    Mac Address      Type      Ports
-----
-----
-----
-----
1       5000.0001.0000   DYNAMIC   Gi0/0
1       5000.0002.0001   DYNAMIC   Gi0/0
Total Mac Addresses for this criterion: 2
```

```
Switch#show spanning-tree
VLAN0001
Spanning tree enabled protocol ieee
Root ID    Priority    32769
Address    5000.0001.0000
Cost       8
Port       1 (GigabitEthernet0/0)
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

Bridge ID  Priority    32769 (priority 32768 sys-id-ext 1)
Address    5000.0003.0000
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
Aging Time 15 sec

Interface Role Sts Cost Prio.Nbr Type
-----
Gi0/0    Root FWD 4    128.1 P2p
Gi0/2    Desg FWD 4    128.2 P2p
Gi0/3    Desg FWD 4    128.4 P2p
Gi1/0    Desg FWD 4    128.5 P2p
```

2. Indirect Topology Changes:

The link failure indirectly involves Switches A and C. The link status on each switch remains up, but something between them has failed or is filtering traffic. This could be another device, such as a service provider's switch, a firewall, or something similar. As a result, no data (including BPDUs) can pass between the two switches.

Q. Why Leaving STP Defaults is a Bad Idea

1. Suboptimal Root Bridge Selection

- Root bridge is elected based on the lowest bridge ID (priority + MAC).
- If priorities are all default (32768), then the switch with the lowest MAC address wins.
- This could accidentally be your slowest switch (e.g., an old access-layer device), not a high-performance distribution/core switch.
- Result: poor performance because all inter-VLAN or inter-segment traffic funnels through a weak switch.

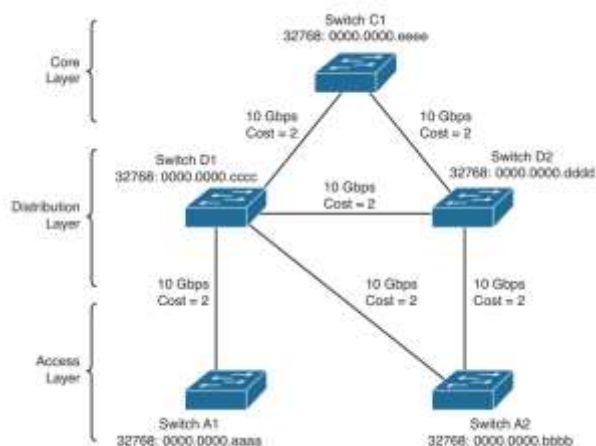
2. No Backup / Poor Redundancy

- Only one root bridge is elected.
- If it fails, another election happens, but the new root is chosen by the same default rules (lowest BID).
- Again, you could end up with an undesirable root location (not a deliberate backup candidate).

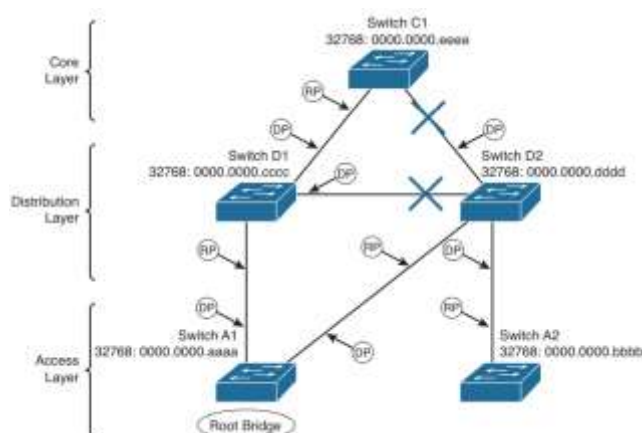
3. Inefficient Topology (Root in the Wrong Place)

- If the elected root bridge is in a corner of the network, traffic between two distant access switches may need to take a long detour through the root.
- This increases latency and link utilization, creating a “winding path” problem.

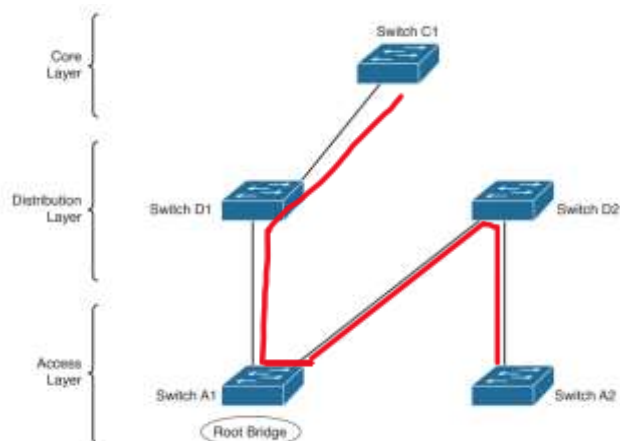
(Below 3 images shows the importance of the root bridge placement)



(topology)



(default root bridge election)



Sub optimal path, ex:- Switch A2 traffic needs to pass through

Sw D2 (Distribution layer) --- Sw A1 (Access layer) --- Sw D1 (Distribution layer) --- Sw C1 (Core layer)

Root Bridge Configuration

To prevent the surprises outlined in the previous section, you should always do two things:

- Configure one switch as a root bridge in a determined fashion.
- Configure another switch as a secondary root bridge, in case of a primary root bridge failure.

As the common reference point, the root bridge (and the secondary) should be placed near the centre of the Layer 2 network. For example, a switch in the distribution layer would make a better root bridge choice than one in the access layer because more traffic is expected to pass through the distribution layer devices.

Bridge ID Formats in STP

1. Traditional 802.1D Format

- Bridge ID (BID) = Priority (16 bits) + MAC address (48 bits)
- The full 64-bit BID is unique because of the MAC address.

Example:

- Priority = 32768
- MAC = 00:11:22:33:44:55
- BID = 32768.0011.2233.4455

2. 802.1t Extended System ID (Modern Default)

- STP was later enhanced to support Per-VLAN STP (PVST+)
- Problem: You'd need a unique MAC address per VLAN if using the traditional method → not scalable.
- Solution: Use the Extended System ID.

BID = (Priority [only upper 4 bits]) + (VLAN ID [12 bits]) + MAC address

- Priority is a multiple of 4096 (4 bits).
- VLAN ID fills the remaining 12 bits of the 16-bit field.
- Example (VLAN 10):
- Priority = 32768 (default)
- VLAN ID = 10
- MAC = 00:11:22:33:44:55
- BID = 32778.0011.2233.4455
(32768 + VLAN 10 = 32778)

3. When Each is Used

- If a switch cannot provide 1024 unique MAC addresses (one per VLAN), then extended system ID is always enabled.
- On modern Catalyst switches, extended system ID is the default

4. Configuration Command

- If you want to enable extended system ID explicitly:
- **Switch(config)# spanning-tree extend system-id**

How to make a switch as root bridge:

You can do this in **two ways**:

Method 1: Manually set the priority

- Default priority = **32768** (or 32768 + VLAN number if extended system ID is used).
- Valid values = **0 to 65535**, but **must be multiples of 4096** if extended system ID is enabled.
- Lower priority = more chance to become root.

Example:

Switch(config)# spanning-tree vlan 2,10-20 priority 4096

This sets VLAN 5 and VLANs 100–200 priority to **4096** (much lower than default), making this switch the likely root.

If you type a wrong number (like 4000), the switch will suggest valid multiples of 4096.

Method 2: Let the switch auto-choose (easy way)

Instead of manually setting numbers, just tell the switch to be root or backup root:

```
Switch(config)# spanning-tree vlan <vlan-id> root {primary | secondary} [diameter <diameter>]
```

- primary → Switch lowers its priority enough to become the root bridge.
- secondary → Switch sets its priority slightly higher than primary, so it becomes backup root if primary fails.

Optional:

```
[diameter number]
```

This tells STP how many switches are in the network path (topology diameter). STP then adjusts its timers automatically.

Remember while setting bridge id using root primary or secondary:

- When we set the root bridge using root primary or secondary command, the primary sets the priority value as 4096, the secondary priority always fixed to 28,172.
problem: if any switch has priority of 4096, its mac-address less than the switch that we want to make root bridge. Device having the lowest mac-address will be choose as root bridge which is not the result we want.

Tuning the Root Path Cost:

Root path cost is the cumulative cost as BPDU travels along. When BPDU is received by a switch port, it adds its own port cost to the root path cost.

$$\text{Port cost} = \frac{1}{\text{int bandwidth}}$$

Port cost can be change using the following command:

```
Switch (config-if)# spanning-tree vlan <vlan-id> cost <cost>
```

You can see the **port cost of an interface** by using the following command:

```
Switch# show spanning-tree interface type member/module/number [ cost]
```

Tuning the Port ID:

Every port on a switch has a **Port ID**, which is a **16-bit number**.

The first 8 bits = Port Priority

The next 8 bits = Port Number

1. Port Priority

- Value: **0 to 255**
- Default: **128**
- You can change it if needed. Lower priority = more chance the port is chosen for STP forwarding.

2. Port Number

- Value: **0 to 255**
- Represents the actual physical port.
- Example: starts at **1/0/1**, then goes up as you move across modules, slots, or stack members.
- Sometimes the numbers don't look "in order" because different modules/stack members have different number ranges.

3. Special Case – EtherChannel

If ports are grouped into an **EtherChannel (Port Channel)**, their **Port ID becomes higher** than if they were used as individual ports.

Command used to check the port id of a switch:

```
Switch# show spanning-tree interface gi0/0
```

We can't change the port no, but we change the port priority for tuning the port-id; With the help of following command, we can change the port priority of a port for a specific vlan.

```
Switch(config)# interface gigabitethernet 0/0
```

```
Switch(config-if)# spanning-tree vlan 10,20 port-priority 64
```

Tuning Spanning-Tree Convergence:

STP uses timers, port states, and rules for topology changes to stop loops in a network. These work with default values made for normal networks. In most cases, the default STP settings are enough to keep the network free of loops and let users communicate.

Sometimes, the default STP settings can slow down network access while waiting for timers to finish, even on links where loops can't happen. For example, if only one PC is connected to a switch port, a loop is not possible. Another case is in large Layer 2 networks, because the default STP timers are set for an average-sized network.

In a smaller network, waiting for the default timer values may be unnecessary because they could be set to shorter times. In such cases, you can adjust the STP settings to make the network converge faster and work more efficiently.

Modifying STP Timers:

STP uses three timers to manage port states and communication between switches. These timers can be changed with specific commands. **They only need to be set on the root bridge**, because it sends the timer values to all other switches in the network through its configuration BPDUs.

```
SW1_primary#show spanning-tree
VLAN0001
  Spanning tree enabled protocol ieee
  Root ID    Priority    4097
            Address    5000.0001.0000
            This bridge is the root
            Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

  Bridge ID  Priority    4097 (priority 4096 sys-id-ext 1)
            Address    5000.0001.0000
            Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
            Aging Time 300 sec
```

```
SW5#show spanning tree
VLAN0001
  Spanning tree enabled protocol ieee
  Root ID    Priority    4097
            Address    5000.0001.0000
            Cost        4
            Port        1 (GigabitEthernet0/0)
            Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

  Bridge ID  Priority    32768 (priority 32768 sys-id-ext 1)
            Address    5000.0005.0000
            Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
            Aging Time 300 sec
```

(default timer: root bridge of Sw1, Sw5 participated in STP)

Ratio of STP timers:

Hello : MaxAge : ForwardDelay = 2 : 20 : 15 = 1 : 10 : 15/2

we need to properly know how to change the timers because each timers depends upon some specification of STP.

Commands used to manually change timers of the root bridge:

```
SW1_primary(config)#spanning-tree vlan 1 hello-time 3
SW1_primary(config)#spanning-tree vlan 1 max-age 30
SW1_primary(config)#spanning-tree vlan 1 forward-time 22
```

Results:

```
SW1_primary#show spanning-tree vlan 1
VLAN0001
  Spanning tree enabled protocol ieee
  Root ID    Priority    4097
            Address    5000.0001.0000
            This bridge is the root
            Hello Time 3 sec Max Age 30 sec Forward Delay 22 sec

  Bridge ID  Priority    4097 (priority 4096 sys-id-ext 1)
            Address    5000.0001.0000
            Hello Time 3 sec Max Age 30 sec Forward Delay 22 sec
            Aging Time 300 sec
```

```
SW5#show spanning-tree
VLAN0001
  Spanning tree enabled protocol ieee
  Root ID    Priority    4097
            Address    5000.0001.0000
            Cost        4
            Port        1 (GigabitEthernet0/0)
            Hello Time 3 sec Max Age 30 sec Forward Delay 22 sec

  Bridge ID  Priority    32768 (priority 32768 sys-id-ext 1)
            Address    5000.0005.0000
            Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
            Aging Time 300 sec
```


Commands used to automatically change timers of the root bridge:

Here we need to give two important terms, 1st is diameter that is maximum number of bridges between any two end nodes, and 2nd one is the hello timer.

Default STP timer is based on diameter 7 and hello-timer 2s.

```
SW1_primary(config)#spanning-tree vlan 1 root primary diameter 3 hello-time 2
SW1_primary(config)#
```

Result: sdf

```
SW1_primary#show spanning-tree vlan 1
```

```
VLAN0001
Spanning tree enabled protocol ieee
Root ID    Priority    4097
Address    5000.0001.0000
This bridge is the root
Hello Time  2 sec  Max Age 12 sec  Forward Delay 9 sec

Bridge ID  Priority    4097  (priority 4096 sys-id-ext 1)
Address    5000.0001.0000
Hello Time  2 sec  Max Age 12 sec  Forward Delay 9 sec
Aging Time  300 sec
```

```
SW5#show spanning-tree
```

```
VLAN0001
Spanning tree enabled protocol ieee
Root ID    Priority    4097
Address    5000.0001.0000
Cost       4
Port       1 (GigabitEthernet0/0)
Hello Time  2 sec  Max Age 12 sec  Forward Delay 9 sec

Bridge ID  Priority    32769  (priority 32768 sys-id-ext 1)
Address    5000.0005.0000
Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
Aging Time  300 sec
```

Redundant Link Convergence:

1. Portfast:

PortFast is a switch feature that lets a port connected to a PC or end device go immediately to Forwarding state, skipping the normal 30-second STP delay (skips listening and learning state). A TCN won't be generated for state changes on a portfast-enabled port.

Example: Suppose a PC, printer, camera, or server is connected to a switch port. If you turn off the PC and later turn it back on, it will take about 30 seconds (sometimes up to 50 seconds) before the port becomes active. This 30-second delay can be skipped with the help of PortFast.

It can be applied **two** ways,

- i. **Globally:** If you enable PortFast globally, then all access ports (non-trunk ports) will automatically be considered as PortFast ports. Command is used:
Switch(config)# spanning-tree portfast default
- ii. **Per port:** You can enable or disable the PortFast feature on a specific switch port by using the following interface configuration command:
Switch(config-if)# [no] spanning-tree portfast

check: show spanning-tree interface g0/2 portfast

When we enable, the following notification generates:

```
SW5(config-if)#spanning-tree portfast
Warning: portfast should only be enabled on ports connected to a single
host. Connecting hubs, concentrators, switches, bridges, etc... to this
interface when portfast is enabled, can cause temporary bridging loops.
Use with CAUTION

Portfast has been configured on GigabitEthernet0/2 but will only
have effect when the interface is in a non-trunking mode.
```

```
SW5(config)#spanning-tree portfast defau
Warning: this command enables portfast by default on all interfaces. You
should now disable portfast explicitly on switched ports leading to hubs,
switches and bridges as they may create temporary bridging loops.
```

2. Uplinkfast:

UplinkFast is a switch feature that speeds up convergence on access-layer switches when an uplink to the distribution layer fails. It allows a switch to immediately switch to a backup uplink without waiting for the normal STP timers (Listening and Learning), reducing network downtime.

Redundant Uplinks: Only one uplink is Forwarding, others are Blocking. If primary uplink fails, the next best uplink immediately takes over.

The switch's **priority is raised to 49,152** to prevent it from becoming root.

MAC Table Updates: Sends **dummy multicast frames** (to 0100.0ccd.cdcd) to inform upstream switches of new paths.

Command: Switch(config)# spanning-tree uplinkfast <max-update-rate pkts-per-second>

```
SW3(config)#int g0/1
SW3(config-if)#shut
SW3(config-if)#
SW3#
*Sep 29 07:30:47.399: %SPAN TREE FAST-7-PORT_FMD_UPLINK: VLAN0001 GigabitEthernet
0/3 moved to Forwarding (UplinkFast).
*Sep 29 07:30:48.035: %SYS-5-CONFIG_I: Configured from console by console
SW3#
SW3#
*Sep 29 07:30:49.373: %LINK-5-CHANGED: Interface GigabitEthernet0/1, changed sta
te to administratively down
*Sep 29 07:30:50.373: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEth
ernet0/1, changed state to down
```

previously, my topology g0/1 port acts as a root port, I configured g0/3 as a uplink fast port, and I changed the state of g0/1 by shutting it down. The uplink port directly moved from block port to forwarding as soon as the root port goes down.

3. Backbonefast:

It allows a switch to bypass the Max Age timer and immediately accept inferior BPDUs. This reduces the time a switch takes to move from the blocking state to the forwarding state in the event of an indirect failure.

Command: SW(config)# Spanning-tree backbonefast

```
SW3(config)#int g0/0
SW3(config-if)#shut
SW3(config-if)#
SW3#
*Sep 29 19:59:10.610: %SPAN TREE FAST-7-PORT_FMD_UPLINK: VLAN0001 GigabitEthernet
0/3 moved to Forwarding (UplinkFast).
*Sep 29 19:59:11.073: %SYS-5-CONFIG_I: Configured from console by console
*Sep 29 19:59:12.573: %LINK-5-CHANGED: Interface GigabitEthernet0/0, changed sta
te to administratively down
*Sep 29 19:59:13.573: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEth
ernet0/0, changed state to down
```

previously, my topology g0/1 port acts as root port, and I changed the state of g0/1 by shutting it down. One of The backbone port directly moved from block port to forwarding as soon as the root port goes down.

Protecting Switch From Unknown BPDU:

There are two ways, we can protect the switch against unexpected BPDU.

1. Root Guard:

After applying Root Guard on a port, if it never receives a superior BPDU, the port works like a normal STP port. Root guard should be enabled on a port where we never expected to see superior BPDU or root bridge for a VLAN. If the port receives superior BPDU, the whole port becomes block, which is called the root-inconsistent state. It must be applied only on designated ports.

Command:

APPLY: SW(conf-if)# spanning-tree guard root

CHECK: SW# show spanning-tree inconsistentports

```
SW5(config)#int g1/2
SW5(config-if)#span
SW5(config-if)#spanning-tree guard
SW5(config-if)#spanning-tree guard root
SW5(config-if)#spanning-tree guard root
SW5(config-if)#
*Sep 30 07:19:47.550: %SPAN TREE-2-ROOTGUARD_CONFIG_CHANGE: Root guard enabled on
port GigabitEthernet1/2.0
```

Here I have applied root guard on port no gigabytethernet g1/2. As soon as I applied, It gets a notification.

```
Switch(config)#spanning-tree vlan 1 priority 4096
Switch(config)#
Switch(config)#
```

Here, I changed the priority of the switch that is connected to SW5, where Root Guard is enabled.

```
SW5#
*Sep 30 07:25:57.400: %SPAN TREE-2-ROOTGUARD_BLOCK: Root guard blocking port Giga
bitEthernet1/2 on VLAN0001
```

The port g1/2 of SW5 goes down or transitioned to root inconsistent state as soon as it receives the superior bpdu.

BPDU GUARD:

It is always used on on ports where **PortFast** is enabled. This protects the network by shutting down the port if another switch is connected, either by mistake or on purpose. A common place to use it is on **access layer ports**, where only computers, printers, or end devices connect. Since BPDUs are not normally expected from these devices, if one is received, BPDU Guard will stop it.

However, BPDU Guard does not protect against loops if a **hub** is connected. This is because a hub does not send BPDUs; it only repeats the traffic it receives. If a hub is connected to two parts of the network at the same time, it could still create a loop even though BPDU Guard is active.

You should **never enable BPDU Guard on uplink ports** that connect to other switches, especially the root bridge. Uplink ports may legitimately receive BPDUs. If BPDU Guard is enabled there, the switch will shut down the port (put it in *errdisable* state) when a BPDU is detected. This would block the uplink connection and prevent the switch from connecting properly to the rest of the network.

Bpduguard can be applied either **globally** or **interface configuration** mode.

i. Global:

SW(config)# spanning-tree portfast bpduguard default

(This command enables the bpdu guard on all the portfast enabled port)

ii. Interface:

SW(config-if)# spanning-tree bpduguard enable

```
SW(config)#int g1/1
SW(config-if)#spanning-tree bpduguard enable
SW(config-if)#
*Oct 1 06:37:24.186: %SPANTREE-2-BLOCK_BPDUGUARD: Received BPDU on port G1/1 w
ith BPDU Guard enabled. Disabling port.
*Oct 1 06:37:24.187: %SW-4-ERR_DISABLE: bpduguard error detected on G1/1, putt
ing G1/1 in err-disable state
*Oct 1 06:37:25.187: %LINEPROTO-5-UPDOWN: Line protocol in Interface GigabitEt
hernet1/1, changed state to down
*Oct 1 06:37:26.197: %LINK-3-UPDOWN: Interface GigabitEthernet1/1, changed stat
e to down
```

Previously, the port g1/1 is portfast enabled port connected to the switch, where any one connect a switch having priority less than the root bridge, and can be elected as a root bridge. I enabled the bpduguard, so that it can prevent rogue switch to become the root bridge.

iii. Check:

SW# show interface status

or

show interface status err-disabled

The port goes down or **err-disabled** state can be **recovered two ways**.

i. Manually

SW(config)# interface g0/1

SW(config-if)# shutdown

SW(config-if)# no shutdown

- This re-enables the port.
- However, if a BPDU is still present, it will go back to err-disable immediately.

ii. Automatic Recovery (Recommended for Access Ports)

You can configure the switch to automatically recover err-disabled ports after a timer.

SW(config)#errdisable recovery cause bpduguard

SW(config)#errdisable recovery interval 100 <seconds - 100>

Protecting Against Sudden Loss of BPDUs:

There are 2 features that help detect or prevent the unexpected loss of BPDUs:

1. LoopGuard
2. Unidirection Link Detection

1. LOOPGUARD:

In STP, some switch ports are kept in the Blocking state to avoid loops. A port is blocked when it's not the root port or the designated port. It stays blocked as long as it keeps receiving BPDUs from the other side.

Sometimes the physical link remains **up**, but the upstream switch stops sending BPDUs.

- Example: STP is disabled on the neighbor switch.
- Example: A software bug or misconfig blocks BPDUs.
- Example: A unidirectional link failure (one direction of fiber works, the other doesn't).

In this case, the local switch sees the interface is still up but no BPDUs arrive. After the Max Age timer expires, the switch assumes no other STP device is on that port. Then it can incorrectly transition the port to Forwarding, which may create a loop.

Loop Guard is a feature that prevents loops in Spanning Tree Protocol by monitoring BPDU activity on **nondesignated ports**, which are usually **blocked** or used as **backup paths**. If the switch detects that BPDUs are no longer being received on such a port, instead of allowing the port to transition into the Forwarding state, Loop Guard places it in a special "loop-inconsistent" state, which effectively keeps it blocked to avoid forming a

loop. Once BPDUs start arriving again, the port automatically recovers and returns to its normal STP operation without requiring manual intervention.

It can be enabled either globally or interface configuration mode:

1. Globally:

```
SW(config)# spanning-tree loopguard default
```

2. Interface:

```
SW(config-if)# spanning-tree guard loop
```

```
Switch(config)#int g0/1
Switch(config-if)#spanning-tree guard root
Switch(config-if)#
*Oct 2 05:40:17.767: %SPANTREE-2-ROOTGUARD_CONFIG_CHANGE: Root guard enabled on
port GigabitEthernet0/1.
```

UDLD:

Sometimes a link may look fine, but traffic flows only in one direction (unidirectional link). This is dangerous for STP because the switch may think the link is safe and forward traffic, which can create loops.

UDLD (Unidirectional Link Detection) is a Cisco feature that detects such one-way failures. It works by sending UDLD frames with port IDs, and the neighbor must echo them back. If no echo is received, the link is considered unidirectional.

Modes of UDLD:

- i. In **Normal mode**, UDLD only logs a message if a unidirectional link is found.
- ii. In **Aggressive mode**, the switch sends UDLD messages every second for 8 seconds, and if no reply is received, the port is shut down (errdisable).

Timers:

The default UDLD message interval is 15 seconds (configurable 1–90 sec). By default, a unidirectional link is detected in about 45 seconds, which is fast enough to act before STP moves a port into the Forwarding state (~50 sec).

It can be configured either globally or locally.

i. Globally

```
SW(config)# udld { enable | aggressive | message time seconds}
```

udld enable – Normal Mode

udld aggressive – Aggressive Mode

ii. Interface

```
SW(config-if)# udld { enable | aggressive | disable}
```

Once a udld port goes into **errdisable state**, you can enable it by using following command:

```
SW# reset
```

BPDU FILTER (Disable STP on a port):

STP normally runs on all switch ports to stop loops, and BPDUs are always sent and received—even on PortFast ports. If you want to stop BPDUs from being sent or received on certain ports, you can use **BPDU filtering**, which disables STP on those ports.

It can be enabled either globally so that all PortFast ports use BPDU filtering, or you can enable it on specific ports. Use it only when you are sure a port connects to a single host and no loops are possible, because disabling STP on a port can be risky.

i. Globally

```
SW(config)# spanning-tree portfast bpdupfilter default
```

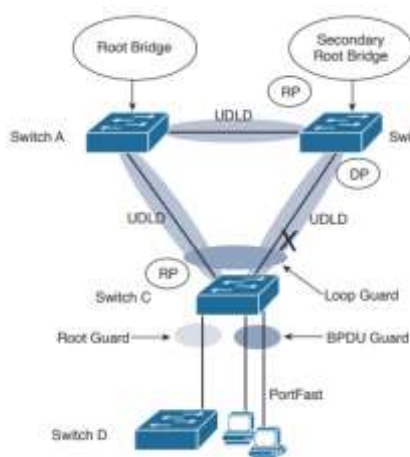
ii. Interface

```
SW(config-if)# spanning-tree bpdupfilter { enable | disable}
```


Important command for troubleshooting:

Display Function	Command Syntax
List the ports that have been labeled in an inconsistent state.	Switch# show spanning-tree inconsistentports
Look for detailed reasons for inconsistencies.	Switch# show spanning-tree interface <i>type mod/num</i> [detail]
Display the global BPDU Guard, BPDU filter, and Loop Guard states.	Switch# show spanning-tree summary
Display the UDLD status on one or all ports.	Switch# show udld [<i>type mod/num</i>]
Reenable ports that UDLD aggressive mode has errdisabled.	Switch# udld reset

Short notes:



Root guard: Apply to ports where root is never expected.

BPDU guard: Apply to all user ports where PortFast is enabled.

Loop guard: Apply to non-designated ports but okay to apply to all ports.

UDLD: Apply to all Fiber-optic links between switches (must be enabled on both ends).

Permissible combinations on a switch port:

Loop guard and UDLD

Root guard and UDLD

Not permissible on a switch port:

Root guard and Loop guard

Root guard and BPDU guard

Task	Global Command Syntax	Interface Command Syntax
Enable Root Guard.	—	Switch(config-if)# spanning-tree guard root
Enable BPDU Guard.	Switch(config)# spanning-tree portfast bpduguard default	Switch(config-if)# spanning-tree bpduguard enable
Enable Loop Guard.	Switch(config)# spanning-tree loopguard default	Switch(config-if)# spanning-tree guard loop
Enable UDLD.	Switch(config)# udld [enable aggressive message time seconds]	Switch(config-if)# udld [enable aggressive disable]
Enable BPDU filtering.	Switch(config)# spanning-tree bpdufilter default	Switch(config-if)# spanning-tree bpdufilter enable

Task	Command Syntax
Look for ports that have been put in an inconsistent state.	Switch# show spanning-tree inconsistentports
Display the global BPDU Guard, BPDU filter, and Loop Guard states.	Switch# show spanning-tree summary
Show UDLD status.	Switch# show udld [<i>type mod/num</i>]
Reenable all ports that UDLD has errdisabled.	Switch# udld reset

```
SW3(config)#spanning-tree portfast edge bpdufilter default
SW3(config)#int g1/1
SW3(config-if)#span
SW3(config-if)#spanning-tree bpdufilter enable
```

In newer IOS images, the commands are slightly different, as shown above.

RSTP (Rapid Spanning-Tree Protocol):

The original 802.1D Spanning Tree Protocol (STP) was designed to prevent loops in switched and bridged networks, but it had one major drawback—convergence was very slow. When a topology change occurred, ports transitioning from a blocking state to a forwarding state could take up to 30 seconds because of the forward delay timers. This delay was acceptable in early networks but became a serious limitation in modern, high-speed environments where downtime of even a few seconds is costly.

To solve this problem, the IEEE introduced 802.1w Rapid Spanning Tree Protocol (RSTP). RSTP retains the same loop-prevention principles as traditional STP but significantly improves convergence, reducing recovery times from 30 seconds to just 1–3 seconds. This makes it much better suited for today's fast networks, especially where redundancy is critical. Importantly, RSTP is backward compatible with classic STP, so it can still interoperate in mixed environments.

RSTP is also the foundation of more advanced implementations. Cisco's Rapid PVST+ (RPVST+) applies RSTP logic on a per-VLAN basis, giving finer control in VLAN-rich networks. Meanwhile, Multiple Spanning Tree (MST, 802.1s) uses RSTP internally to manage multiple spanning-tree instances more efficiently. Together, these protocols ensure both loop prevention and rapid recovery, making RSTP the standard choice in modern enterprise networks.

Port Roles in RSTP:

The root bridge in a network using RSTP is elected just as with 802.1D—by the lowest bridge ID. After the election, it maintains an additional port state that acts as alternate or backup path, the port roles are:-

- i. **Root port** – identical to CST
- ii. **Designated Port** – identical to CST
- iii. **Backup Port** – It is a backup path to a segment where another bridge port is already connected. These ports receive BPDUs from their switches but they remain in a blocked state.
- iv. **Alternate Port** – It is a Backup port with a less desirable path cost. All such ports remain in a blocked state.

Port States in RSTP:

RSTP defines port states only according to what the port does with incoming frames. There are 3 states in rstp:-

Discarding – Incoming frames simply are dropped; no MAC addresses are learned.

Learning – Incoming frames are dropped, but MAC addresses are learned.

Forwarding – Incoming frames are forwarded according to MAC addresses that have been learned.

BPDUs in RSTP:

In 802.1D, BPDUs originate only from the root bridge and are relayed through the tree, causing slower convergence. RSTP improves this by sending BPDUs from every port at each hello interval, allowing all switches to actively maintain the topology and **quickly detect failures within three hello intervals (6 seconds)**, instead of waiting for the 20-second Max Age timer in 802.1D. RSTP uses the same BPDU format for backward compatibility but sets the version to 2 and uses previously unused bits for negotiation, enabling faster state changes between neighbors. It can **coexist with 802.1D switches by adapting to the BPDU version received**, and during migration, a delay timer prevents protocol flapping when both BPDU types are present.

A RSTP enabled switch can be acted as both rstp and cst.

BPDU version 0 – CST

BPDU version 2 – RSTP

RSTP Convergence:

Convergence as a twostage process:

1. One common root bridge must be "elected," and all switches must know about it.
2. The state of every switch port in the STP domain must be brought from a Blocking state to the appropriate state to prevent loops

Port Types:

RSTP defines three types of ports to build and maintain the STP topology:

1. Edge Port:

Connects directly to an end host (like PCs or servers) at the edge of the network. Equivalent to STP PortFast — immediately transitions to the Forwarding state since no loop can form. If a BPDU is received on an edge port, it instantly loses edge status and participates in STP.

2. Non-edge port:

It is a broad category in RSTP. It simply means any port that is not an edge port, i.e., it participates in spanning tree and receives BPDUs.

3. Point-to-Point Port:

A port that connects to another switch via a full-duplex link. Uses a **rapid handshake mechanism** (Proposal and Agreement) instead of timers for fast convergence. Full-duplex links are treated as point-to-point since only two devices exist on the link. **Half-duplex** links are treated as **shared segments**, requiring the slower 802.1D-style convergence with Listening and Learning states.

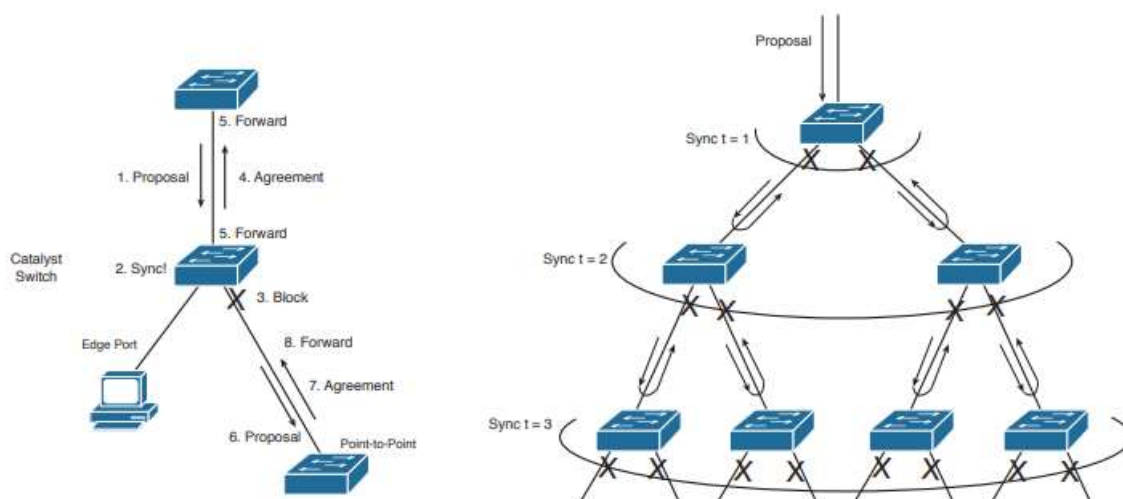
RSTP Topology Building and Synchronization:

Rapid Spanning Tree Protocol (RSTP) allows switches to converge faster than traditional STP by using a proposal-agreement handshake mechanism. This process ensures that switches join the network topology safely without creating forwarding loops.

When two switches first connect, they verify that the link is point-to-point by checking full-duplex status. They then exchange BPDUs, proposing that their interface should become the Designated Port (DP) for the segment. Only one DP exists per segment, so switches compare their system identifiers (priority + MAC address) to determine superiority. The superior switch becomes the designated switch, and the inferior switch designates its port as the Root Port (RP).

For each switch, non-edge ports initially start in the Discarding state to prevent bridging loops. The synchronization process proceeds as follows:

1. **Proposal Message Sent:** A switch assumes its port should be the DP and sends a proposal BPDUs to its neighbor.
2. **Reception of Proposal:** If a switch receives a proposal from a superior neighbor, it recognizes that the neighbor should be the DP, and its own port must become the RP.
3. **Synchronization:** Before agreeing, the switch synchronizes itself with the topology by moving all non-edge ports into the Discarding state, isolating itself to prevent loops.
4. **Agreement Message:** The switch sends an agreement BPDUs back to the neighbor, confirming acceptance of the DP choice and signaling that it is synchronizing.
5. **Forwarding State Transition:** The RP on the local switch immediately transitions to Forwarding, and the DP on the neighbor switch also begins forwarding.
6. **Propagation to Other Ports:** The switch sends proposal messages on all remaining non-edge ports. Agreements are received from neighbors, and those ports are moved to Forwarding as well.
7. **Downstream Synchronization:** This process repeats for any downstream switches, creating a “wave of synchronization” through the network. Each switch only begins forwarding on links after neighbors agree, preventing forwarding loops. (~ recursion)



Sequence of Events During RSTP Convergence

RSTP Synchronization Traveling Through a Network

Topology Changes In RSTP:

In 802.1D STP, when a port goes up or down, the switch notifies the root bridge by sending a Topology Change Notification (TCN) BPDU. The root bridge then informs all other switches in the network about the change so they can update their forwarding tables.

In RSTP, topology changes are detected differently. A change is noticed only when a non-edge port moves to the Forwarding state, because RSTP's fast convergence prevents loops. This ensures that bridging tables are updated only when hosts appear on a new port, for example, after a link failure.

When a topology change occurs, the switch sends TC BPDUs out all its non-edge designated ports. These messages travel through the network like a wave, notifying other switches of the change. Each switch that

receives a TC message:

1. Clears MAC addresses learned on all ports except the one that received the message.
2. Forwards the TC message out its non-edge designated ports to neighbors.

This continues until the TC timer expires (after two Hello intervals). The process ensures that all switches relearn host locations and keep the network forwarding correctly.

RSTP Configuration:

To enable Rapid Spanning Tree Protocol (RSTP) on a Cisco switch, use the following **global configuration** command:

```
SW(config)# spanning-tree mode rapid-pvst
```

To configure a port as an RSTP **edge port**, use the following interface configuration command:

```
SW(config-if)# spanning-tree portfast
```

Point-to-point – Full duplex – (Two switches)

To **force the port to act as a point-to-point link**, use the following interface configuration command:

```
Switch(config-if)# spanning-tree link-type point-to-point
```

Multiple Spanning Tree Protocol:

Why MST Was Developed

Traditional STP had two main versions in use:

- **802.1Q CST (Common Spanning Tree):**

Runs **only one STP instance** for the entire network, no matter how many VLANs exist.

Example: 500 VLANs – just 1 STP instance.

Problem: only one loop-free path exists at a time, so if an access switch has two uplinks, only one forwards and the other always blocks. This wastes bandwidth.

- **PVST+ (Per-VLAN STP):**

Runs **one STP instance per VLAN**.

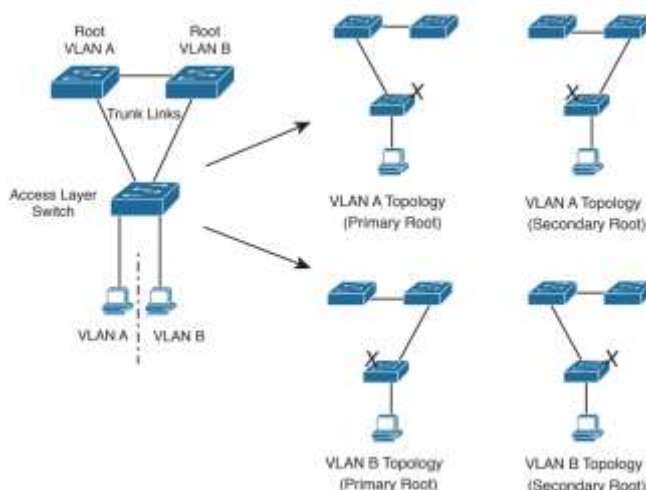
Example: 500 VLANs – 500 STP instances.

Benefit: You can load balance by making different VLANs use different root bridges, so both uplinks can forward traffic.

Problem: Consumes a lot of CPU and memory, especially when the number of VLANs grows large, even though most VLANs don't need unique topologies.

Solution:

MST lets you decide exactly how many spanning-tree instances to run, no matter how many VLANs exist.



MST (Multiple Spanning Tree) works by grouping VLANs together and mapping them to a single STP instance. Instead of running one instance per VLAN (like PVST+), MST allows you to run only the number of instances you actually need.

- You can create multiple MST instances (MSTIs), and each instance can control a different group of VLANs.
- Each MST instance can be tuned to produce a different topology, which allows for load balancing across redundant links.

Example:

In a network with two uplinks:

- Instance 1 can be tuned so it forwards traffic on the left uplink.
- Instance 2 can be tuned so it forwards traffic on the right uplink.
- VLAN A can be mapped to Instance 1, and VLAN B to Instance 2.

This way, both uplinks are used efficiently. When configuring MST, you must decide:

1. How many spanning-tree instances are needed.
2. Which VLANs should be mapped to each instance.

MST Regions:

MST works by grouping switches into **regions** so they can all run the same MST configuration together. For switches to be in the same MST region, they must have three things identical:

1. Configuration name (up to 32 characters)
2. Configuration revision number (0–65535)
3. Instance-to-VLAN mapping table (decides which VLAN belongs to which MST instance)

Above one in sentence: If two switches have the same MST configuration attributes (name, revision number, VLAN-to-instance mapping), they are in the same MST region and can share the same STP instances.

If the attributes don't match, they are considered part of different regions. The point where two regions meet is called the **MST region boundary**. This can also happen when MST connects to a traditional STP (802.1D) switch.

MST BPDUs carry these configuration attributes so switches can compare them. **But instead of sending the full VLAN mapping table (which could be huge), they send a digest (hash value) of the table.** If the digest matches, the mappings are the same. If not, the switches know they are in different regions.

Spanning-Tree Instances Within MST:

In Multiple Spanning Tree (MST), different VLANs can be grouped and mapped to their own spanning-tree instances. However, MST also needs to interoperate with other STP types (like CST, PVST+, RSTP). To achieve this, the network is viewed as having a single Common Spanning Tree (CST) that ties everything together. This CST acts as the overarching loop-free topology for the entire enterprise network, regardless of how many VLANs or MST regions exist inside.

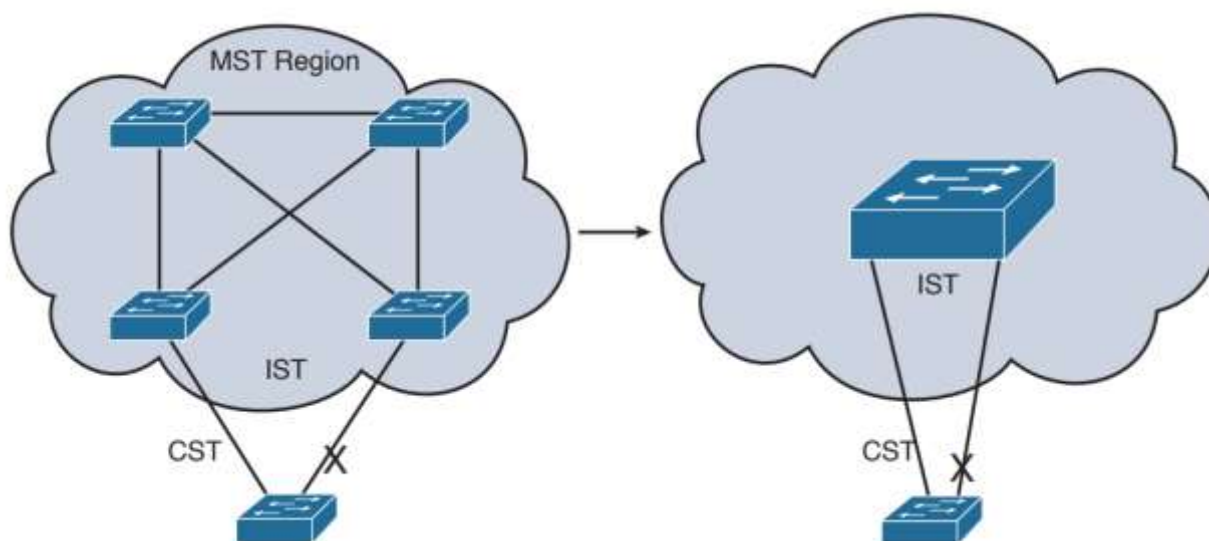
From CST's perspective, each MST region is treated like a single logical bridge or switch. CST doesn't look inside the region—it only cares about the links between MST regions and any standalone switches running CST. This ensures that the loop-free topology is preserved across the whole network, even when multiple STP types and MST regions are mixed together.

IST Instances:

Inside an MST region, the CST outside only sees the region as one logical switch. But something still needs to handle spanning-tree inside the region, making sure that all the switches within the region stay loop-free. This job is done by the Internal Spanning Tree (IST).

The IST is like a locally significant CST that runs only inside the region. It ensures a loop-free topology between the boundary switches (where CST connects) and all internal switches in the region. Once the IST converges, it presents the whole MST region as a single virtual bridge to the outside CST.

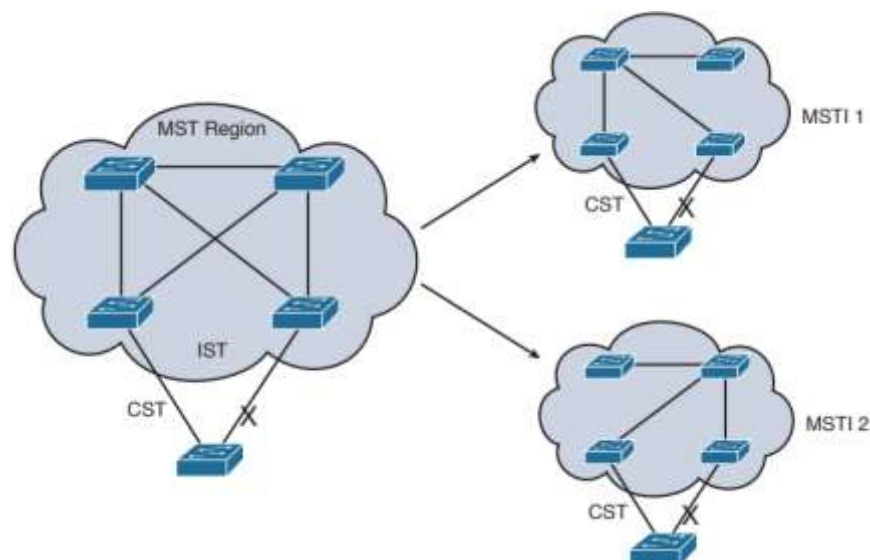
To the CST outside, it looks like it's talking to just one big switch. BPDUs are exchanged only at the region boundaries, and always over the native VLAN on trunks. This keeps the spanning-tree communication simple, even though inside the region multiple switches and MST instances may exist.



MST Instances:

The main idea of MST (Multiple Spanning Tree) is that you don't need to run one STP instance for every VLAN. Instead, you can map many VLANs to fewer STP instances.

- Inside an MST region, there are two kinds of instances:
 - **IST (Internal Spanning Tree)** — always exists as **MST Instance 0**. It keeps the region loop-free and represents the region as one switch to the outside CST.
 - **MSTIs (Multiple Spanning Tree Instances)** — these are the instances you create for load balancing.
- Cisco allows up to 16 instances per region:
 - MSTI 0 = IST (always there).
 - MSTI 1–15 = available for mapping VLANs.



Here, within the same MST region, we have two different instances. Each instance maintains its own logical topology to form the loop-free topology. For the entire region, one instance is present, which is responsible for behaving this entire region as a single logical switch.

MST Configuration:

This passage describes the **configuration process for Multiple Spanning Tree (MST)**, which is part of the IEEE 802.1s standard and allows you to map multiple VLANs to a smaller number of spanning-tree instances — improving scalability and efficiency compared to PVST+.

Here's a **simplified summary of the configuration steps:**

1. Enable MST mode

SW(config)# spanning-tree mode mst

This switches the spanning tree operation from PVST+/RSTP to MST.

2. Enter MST configuration mode

SW(config)# spanning-tree mst configuration

3. Define the MST region name

SW(config-mst)# name REGION_NAME

All switches in the same MST region **must have the exact same name**.

4. Set the revision number

SW(config-mst)# revision <number>

Revision number Used to track configuration changes (0–65535).

It Must match across all switches in the region.

5. Map VLANs to MST instances

SW(config-mst)# instance <instance-id> vlan <vlan-list>

Example:

instance 1 vlan 10,20,30

instance 2 vlan 40-50

VLANs not explicitly mapped remain in Instance 0 (IST) by default.

6. Verify pending configuration

SW(config-mst)# show pending

7. Exit and apply configuration

SW(config-mst)# exit

After exiting, MST becomes active, and PVST+ is automatically disabled — they cannot run simultaneously.

8. check mst configuration

SW# show spanning-tree mst

NOTE:

MST interacts with Common Spanning Tree (CST) or legacy 802.1D using the Internal Spanning Tree (IST). MST tuning commands (timers, priorities, etc.) are similar to RSTP/STP, but they apply to MST instances instead of VLANs. Timers apply globally to the MST region, not individually per instance

```
SW1#show spanning-tree mst 1
----- MST1 -----
Bridge ID: 1
address 5000.0001.0000
Root ID: 1
this switch for MST1

Interface      Role Sts Cost      Prio.Nbr Type
-----
Gi0/0          Desg Fwd 20000    128.1   P2p
Gi0/1          Desg Fwd 20000    128.2   P2p
Gi0/2          Desg Fwd 20000    128.3   P2p
Gi0/3          Desg Fwd 20000    128.4   P2p
Gi1/0          Desg Fwd 20000    128.5   P2p
Gi1/1          Desg Fwd 20000    128.6   P2p
Gi1/2          Desg Fwd 20000    128.7   P2p
Gi1/3          Desg Fwd 20000    128.8   P2p

SW1#show spanning-tree mst int gi0/0
GigabitEthernet0/0 of MST0 is designated forwarding
Portfast : no (default) port guard : none (default)
Link type : point-to-point (auto) bpdu filter : disable (default)
Boundary : internal bpdu guard : disable (default)
PVST 5in : enable (default)
Bpdus sent 1515, received 522

Instance Role Sts Cost      Prio.Nbr Vlan mapped
-----
0        Desg Fwd 20000    128.1   101-4094
1        Desg Fwd 20000    128.1   1-30
2        Root Fwd 20000    128.1   51-100

SW1#show spanning-tree mst configuration
Name: [ATG1]
Revision 2 Instances configured 3

Instance Vlan mapped
-----
0        101-4094
1        1-30
```

Global config mode commands...

- spanning-tree vlan *vlan* root {primary | secondary}
- spanning-tree vlan *vlan* priority *priority*
- spanning-tree mst instance root {primary | secondary}
- spanning-tree mst instance priority *priority*
- spanning-tree vlan *vlan* hello-time *seconds*
- spanning-tree mst hello-time *seconds*
- spanning-tree vlan *vlan* forward-time *seconds*
- spanning-tree mst forward-time *seconds*
- spanning-tree vlan *vlan* max-age *seconds*
- spanning-tree mst max-age *seconds*

Q. why don't we need to specify the instance in this commands?

Because mst instance 0 is responsible to send BPDU messages outside regions.

DTP (Dynamic Trunking Protocol):

- DTP is used to auto negotiate the trunk ports.
- DTP can also negotiate whether a port becomes a trunk at all.
- DTP has two modes to dynamically decide whether a port becomes a trunk:
Desirable – the port will actively attempt to form a trunk with the remote switch. This is the default setting.
Auto – the port will passively wait for the remote switch to initiate the trunk.

Commands:

switchport mode dynamic auto

switchport mode dynamic desirable

Troubleshooting Trunk Connections A trunk connection requires several parameters to be configured identically on both sides of the trunk:

- Trunk
- Frame-tagging protocol
- Native VLAN
- Allowed VLANs
- VTP Domain – only when using DTP to negotiate a trunk

802.1Q embeds a 4-byte VLAN tag directly into the Layer-2 frame header. Because the Layer-2 header is modified, 802.1Q must recalculate the frame's CRC value.

VLAN Trunking Protocol (VTP):

Maintaining a consistent VLAN database can be difficult in a large switch environment.

VTP (VLAN Trunking Protocol) helps by automatically sharing VLAN updates with all switches in the same VTP domain through VTP advertisements and a switch can only belong to a single domain.

VTP versions:

i. V1 – support 1005 vlan

ii. V2 – support token ring, Vlan consistency checks, Domain-independent pass through

If the VTP server is configured for VTPv2, all other switches in the VTP domain will change to v2 as well.

iii. V3 – supports 4095 vlan, mst

Other enhancements provided by VTPv3 include:

- Support for the extended 1006-4094 VLAN range.
- Support for private VLANs.
- Improved VTP authentication.
- Protection from accidental database overwrites, by using VTP primary and secondary servers.
- Ability to enable VTP on a per-port basis.

VTP modes:

- Server - responsible for creating, deleting, or modifying entries in the VLAN database.
- Client – it can't create and modify, learns VLAN database from VTP server, other clients, and it will also forward the VTP advertisements out every trunk port.
- Transparent - maintains its own local VLAN database, and does not directly participate in the VTP domain. A transparent switch will never accept VLAN database information from another switch, even a server. Also, a transparent switch will never advertise its local VLAN database to another switch. Transparent switches will pass through advertisements from other switches in the VTP domain.

The VTP version dictates how the pass through is handled:

- VTP version 1 – the transparent switch will only pass through advertisements from the same VTP domain.
- VTP version 2 – the transparent switch will pass through advertisements from any VTP domain.

VTP Advertisements – Revision Number

updates to the VLAN database are propagated using VTP advertisements. VTP advertisements are always sent out trunk ports, on VLAN 1. VTP advertisements are marked with a 32-bit configuration revision number, to identify the most current VLAN database revision. Any change to the VLAN database increments the configuration revision number by 1. Thus, a higher number represents a newer database revision. A switch will only accept an advertisement if the revision number is higher than the current VLAN database.

Advertisements with a lower revision number are ignored.

There are two methods of **resetting the revision number** to zero on a switch:

1. Change the VTP domain name, and then change it back to the original name.
2. Change the VTP mode to transparent, and then change it back to either server or client. Transparent switches always a revision number of 0.

Page:87

VTP Advertisements – Message Types Three message types exist for VTP advertisements:

- **Summary Advertisement**
- **Subset Advertisement**
- **Advertisement Request**

Both VTP servers and clients will send out a summary advertisement every 300 seconds.

Summary advertisements contain the following information about the VTP domain:

- VTP version
- Domain name
- Configuration revision number
- Time stamp – indicates which vlan information is more recent
- MD5 digest Summary advertisements are also sent when a change occurs to the VLAN database.

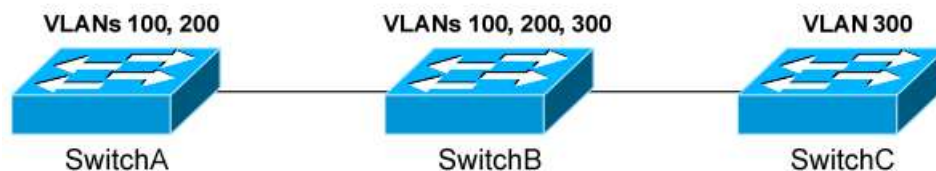
The summary is then followed with a **subset advertisement**, which actually contains the full, updated VLAN database. A subset advertisement will contain the following information:

- VTP version
 - Domain name
 - Configuration revision number
 - VLAN IDs for each VLAN in the database
 - VLAN-specific information, such as the VLAN name and MTU
- Important note: Switches will only accept summary and subset advertisements if the domain name and MD5 digest match. Otherwise the advertisements are ignored.

NOTE: If a switch receives a summary advertisement with a revision number higher than its own, it will send out an advertisement request. VTP servers will then respond with an updated summary and subset advertisement so that the switch can synchronize to the most current VLAN database.

VTP pruning

A Layer-2 switch will thus forward both broadcasts and multicasts out every port in the same VLAN but the originating port. This includes sending out broadcasts out trunk ports to other switches, which will in turn flood that broadcast out all ports in the same VLAN. VTP pruning eliminates unnecessary broadcast or multicast traffic throughout the switching infrastructure.



VTP pruning allows a switch to learn which VLANs are active on its neighbours. Thus, broadcasts are only sent out the necessary trunk ports where those VLANs exist.

VTP pruning is disabled by default on IOS switches. VTP pruning must be enabled on a server, and will be applied globally to the entire VTP domain:

Switch(config)# **vtp pruning**

Both VLAN 1 and the system VLANs 1002-1005 are never eligible for pruning. To manually specify which VLANs are pruning eligible on a trunk:

Switch(config)# interface gi2/24

Switch(config-if)# switchport trunk pruning vlan 2-10

Switch(config-if)# switchport trunk pruning vlan add 42

Switch(config-if)# switchport trunk pruning vlan remove 5

Switch(config-if)# switchport trunk pruning vlan except 100-200

Switch(config-if)# switchport trunk pruning vlan none

Etherchannel:

Spanning Tree Protocol (STP)

- Invented by: Radia Perlman
- Year: 1985
- Purpose: Prevent loops in Layer 2 networks by creating a loop-free logical topology.

EtherChannel

- Introduced by: Kalpana, later acquired by Cisco
- Year: Early 1990s (around 1994)
- Purpose: Bundle multiple physical links into one logical link for increased bandwidth and redundancy.

There are two issues with using only a single physical port for the trunk connection:

- The port represents a **single point of failure**. If the port goes down, the trunk connection is lost.
- The port represents a **traffic bottleneck**. All other ports on the switch will use that one port to communicate across the trunk connection.

However, simply trunking two or more ports between the switches will not work, as this creates a switching loop. One of two things will occur:

- Spanning Tree Protocol (STP) will disable one or more ports to eliminate the loop.
- If STP is disabled, the switching loop will result in an almost instantaneous broadcast storm, crippling the network.

Port Aggregation:

It is used to bind multiple physical ports to a single logical port. The switch and STP treats it as a single port. It supports 8 ports in a single etherchannel.

Historically, port-security has not been supported on an EtherChannel. Newer platforms may provide support as long as port-security is enabled on both the physical interfaces and the EtherChannel itself.

EtherChannel Load-Balancing Traffic sent across an EtherChannel is not evenly distributed across all ports in the bundle. Instead, EtherChannel utilizes a load-balancing algorithm to determine the port to send the traffic out, based on one of several criteria:

- Source IP address - src-ip
- Destination IP address - dst-ip

- Source and destination IP address - src-dst-ip
- Source MAC address - src-mac
- Destination MAC address - dst-mac
- Source and Destination MAC address - src-dst-mac
- Source TCP/UDP port number - src-port
- Destination TCP/UDP port number - dst-port
- Source and destination port number - src-dst-port

The load-balancing method must be configured globally on the switch:

Switch(config)# port-channel load-balance src-dst-mac

To display the currently configured load-balancing method:

Switch# show etherchannel load-balance

To view the load on each port in an EtherChannel (output abbreviated): Switch# show etherchannel 1 port-channel

Example

You have two source IP addresses:

1. 10.1.1.2 → binary: 00001010.00000001.00000001.00000010
2. 10.1.1.3 → binary: 00001010.00000001.00000001.00000011

Just focus on the **last octet** (last 8 bits), which is:

- .2 → 00000010
- .3 → 00000011

The Load-Balancing Logic

If the EtherChannel has **2 physical links**, it only needs **1 bit** (because 2 links = 2 possibilities = 1 binary bit).

- A **bit** is either 0 or 1.
- The algorithm might just look at the **last bit** of the hash result or IP.

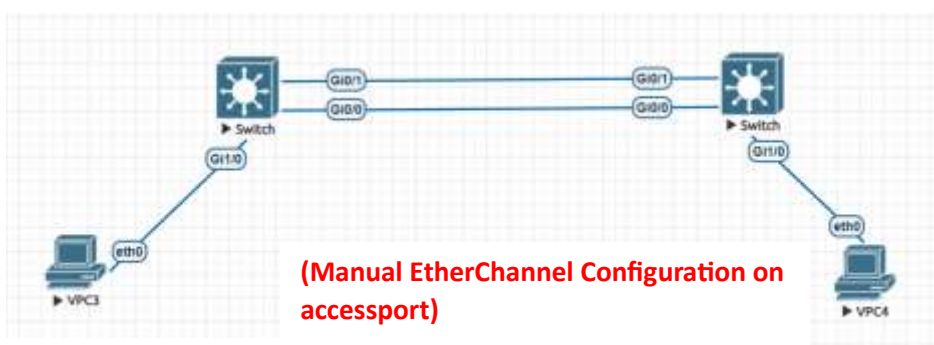
So:

- 00000010 (last bit = 0) → send to **Link0**
- 00000011 (last bit = 1) → send to **Link1**

Ether Channel Configuration:

i. manual

ii. Dynamic - using an aggregation protocol



LEFT_Switch:

```

en
conf t
int r gi0/0 - 1
switchport mode access
channel-group 1 mode on
exit
! Configure Port-Channel
interface port-channel 1
switchport mode access
sw ac vl 10
exit
! Configure access port to PC
interface gi1/0
switchport mode access
switchport access vlan 10
exit

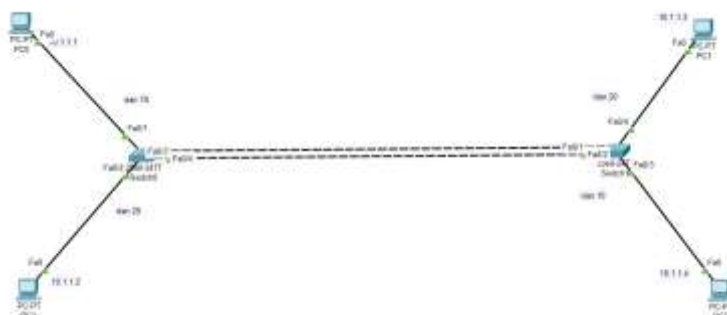
```

right_switch:

```

en
conf t
int r gi0/0 - 1
switchport mode access
channel-group 1 mode on
exit
! Configure Port-Channel
interface port-channel 1
switchport mode access
sw ac vl 10
exit
! Configure access port to PC
interface gi1/0
switchport mode access
switchport access vlan 10
exit

```



LEFT_Switch:

```

en
conf t
interface range gi0/0 - 1
switchport mode trunk
sw mo dy desirable
channel-group 1 mode on
exit

```

```

interface port-channel 1
switchport mode trunk
exit

```

Right_Switch:

```

en
conf t
interface range gi0/0 - 1
switchport mode trunk
sw mo dy auto
channel-group 1 mode on
exit

```

```

interface port-channel 1
switchport mode trunk
exit

```

EtherChannel:

Dynamic Configuration Cisco switches support two dynamic aggregation protocols:

PAGP (Port Aggregation Protocol)	LACP (Link Aggregation Protocol)
Cisco proprietary	Open standard
Modes (Desirable, Auto)	Modes (Active, Passive)
8 links in one channel	16 links in one channel
8 links are active	8 links are active; 8 links are backup

LACP supports adding an additional 8 ports into the bundle in a standby state, to replace an active port if it goes down.

- **Desirable/Active** – actively attempts to form a channel
- **Auto/Passive** – waits for the remote switch to initiate the channel

the EtherChannel using **PAGP** negotiation:

```

Switch(config)# interface range g 0/0-1
Switch(config-if)# channel-protocol pagp
Switch(config-if)# channel-group 1 mode desirable
Switch(config-if)# channel-group 1 mode auto

```

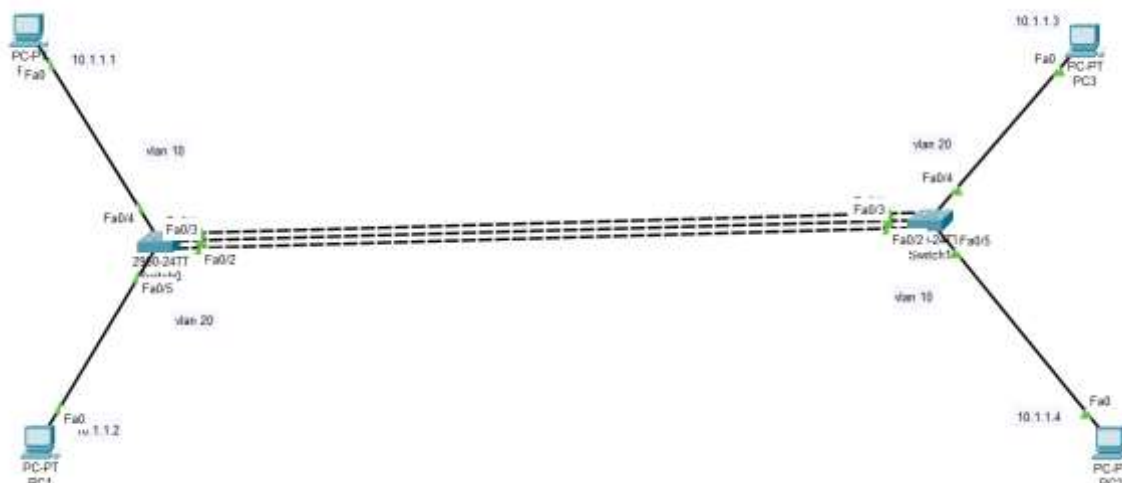
the EtherChannel using **LACP** negotiation:

```

Switch(config)# interface range g 0/0-1
Switch(config-if)# channel-protocol lacp
Switch(config-if)# channel-group 1 mode active
Switch(config-if)# channel-group 1 mode passive

```

PAGP (Port Aggregation Protocol)



Left_Switch:

```

en
conf t
!
int f0/4
sw ac vl 10
int f0/5
sw ac vl 20
!
int r f0/1-3
channel-protocol pagp
channel-group 1 mode desirable

```

Right_Switch:

```

en
conf t
!
int f0/4
sw ac vl 20
int f0/5
sw ac vl 10
!
int r f0/1-3
channel-protocol pagp
channel-group 1 mode desirable

```